

Security for Connected Objects

Nancy AWAD

nancy.awad@univ-fcomte.fr

International Master in Computer Science

September - October 2025



Contents

1	Introduction	5
1.1	Connected Objects and the Internet of Things	5
1.1.1	From the Internet of Content to the Internet of Things	5
1.1.2	What is IoT?	5
1.1.3	IoT Architecture and Components	6
1.1.4	Industry Applications of IoT	7
1.2	Overview of an IoT Ecosystem	8
1.2.1	Functional Building Blocks of IoT	8
1.2.2	Non-functional Challenges of IoT Systems	8
1.2.3	Benefits and Risks of IoT	9
1.2.4	The computational layers in IoT systems	10
1.3	Threats in the Era of Machine-to-Machine Communication . .	11
1.4	Activity	12
2	Introduction to Cybersecurity	13
2.1	Cybersecurity vs Computer Security	13
2.2	Core Security Concepts: A Chain of Events	14
2.3	Challenges in Security	14
2.4	The CIA triad	14
2.5	Layers of Security Vulnerabilities	16
2.5.1	Hardware Vulnerabilities:	16
2.5.2	Software Vulnerabilities:	16
2.5.3	Data Vulnerabilities:	16
2.5.4	Network Vulnerabilities:	17
2.6	Elements of Cybersecurity	17
2.7	Security Laws	18
2.8	Discussions	19
3	Cyberattacks	20
3.1	Passive attacks	20
3.1.1	Eavesdropping attack	20
3.1.2	Traffic Analysis	21
3.2	Active Attacks	21
3.2.1	Man-in-the-Middle Attacks:	21
3.2.2	SQL injection	21
3.3	Insider Attacks	22
3.4	Outsider Attacks	22
3.5	Distributed Attacks	23
3.6	Social Engineering Attacks	23

3.7	Malware Attacks	26
3.7.1	Viruses	26
3.7.2	Worms	26
3.7.3	Trojan Horse	26
3.8	Physical Attacks	27
3.9	Password cracking Attacks	27
3.9.1	Brute Force Attacks	27
3.9.2	Credential Stuffing	28
3.10	Advanced Persistent Threats (APTs)	28
4	Network security and attacks	29
4.1	Elements of a network	29
4.1.1	Network Architecture	29
4.1.2	Devices in a network	29
4.1.3	Protocols	29
4.1.4	IP Addresses and DNS	30
4.1.5	Network Traffic	30
4.1.6	Internet Control Message Protocol (ICMP)	30
4.2	Network security policies	31
4.2.1	Firewalls	31
4.2.2	Password policies	32
4.2.3	Intrusion detection systems	32
4.2.4	Anti-malware tools	32
4.2.5	Virtual Private Networks (VPNs)	33
4.2.6	Authentication Policies	33
5	History of Cryptographic tools	35
5.1	Ancient Encryption and Decryption:	35
5.2	Spartan Scytale (c. 400 BCE):	35
5.3	Caesar Cipher (c. 58–50 BCE):	35
5.4	Middle Ages and Renaissance (500–1500 CE)	36
5.5	Vigenère Cipher (16th Century)	36
5.6	Early Modern Period (1500–1800)	36
5.7	Charles Babbage and the Vigenère Cipher (1854):	37
5.8	World War I (20th Century)	38
5.9	World War II and Enigma (1930s–1940s):	38
5.10	Post-War and Digital Age (1940s–present)	38
5.11	Public-Key Cryptography (1970s):	38
5.12	Modern Cryptanalysis and Quantum Threats:	39

6	Cryptography	40
6.1	Encryption	40
6.1.1	Modular Addition	40
6.1.2	Modular Multiplication	41
6.1.3	Modular Exponentiation	42
6.1.4	Example to compare the three methods	43
6.1.5	Key VS modulo: a distinction to make	45
6.2	How to Choose the Modulus	46
6.2.1	Fixed Modulus in Classical Systems	46
6.2.2	Larger Moduli in Modern Cryptography	46
6.2.3	Balancing Security and Efficiency	47
6.3	Security through complexity	47
6.4	Symmetric and asymmetric encryption	47
6.5	Symmetry and Asymmetry in modular encryption	49
6.6	Symmetric Encryption Algorithms	50
6.6.1	AES (Advanced Encryption Standard)	50
6.6.2	DES (Data Encryption Standard) and 3DES	51
6.6.3	3DES (Triple DES)	51
6.6.4	Applications of Symmetric Encryption	51
6.7	Asymmetric encryption Algorithms	52
6.7.1	RSA (Rivest-Shamir-Adleman)	52
6.7.2	ECC (Elliptic Curve Cryptography)	52
6.7.3	Applications of Asymmetric Encryption	53
6.8	Hashing functions	54
6.8.1	How Hashing Works?	54
6.8.2	Role of Hashing in Cybersecurity	54
6.8.3	MD5 (Message-Digest 5) Hashing Function	55
6.8.4	SHA (Secure Hash Algorithm)	56
6.9	Key Exchange Protocols for Symmetric Encryption	57
6.9.1	Diffie-Hellman (DH)	57
6.9.2	Elliptic Curve Diffie-Hellman (ECDH)	59
6.10	Key Exchange Protocols for Asymmetric Encryption	60
6.10.1	Exchanging modulus with public keys?	61
7	Security for IoT	62
7.1	Security challenges for IoT	63
7.2	IoT Security Vulnerabilities Layers	65
7.2.1	Hardware vulnerabilities	65
7.2.2	Software vulnerabilities	66
7.2.3	Network vulnerabilities	66
7.3	Security Considerations for IoT	67

7.4	IoT hardware and software	73
7.5	IoT Communication and Messaging Protocols	75
7.6	Threats, Vulnerabilities and Risks in IoT	76
7.7	IoT Security Architecture	77
7.8	Lightweight Encryption Algorithms for IoT	78
7.8.1	Block Ciphers - "Encrypt in chunks"	78
7.8.2	Stream Ciphers - "Encrypt as you go"	79
7.8.3	AEAD (Authenticated Encryption with Associated Data) - "Encrypt and verify"	79

1 Introduction

1.1 Connected Objects and the Internet of Things

1.1.1 From the Internet of Content to the Internet of Things

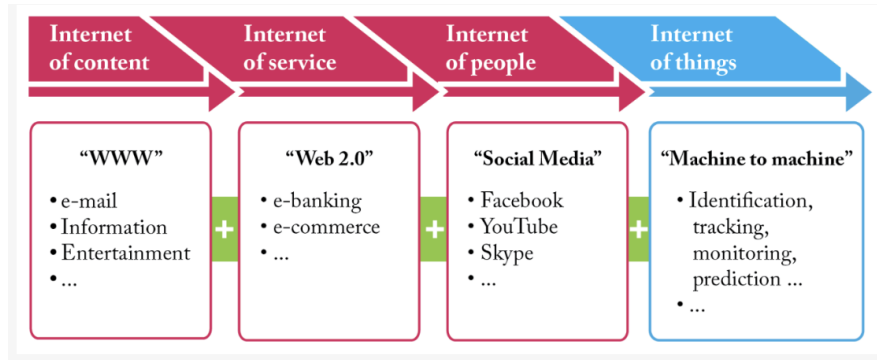


Figure 1: Evolution of the internet

The Internet has undergone successive transformations.

1. In its early stages, the *Internet of Content* was primarily about information exchange through the World Wide Web, e-mail, and digital libraries.
2. This gave way to the *Internet of Services*, enabling online commerce, banking, and interactive platforms.
3. The next evolution was the *Internet of People*, marked by the rise of social networks, video sharing, and real-time communication.
4. Today, we are in the era of the *Internet of Things* (IoT), where not only humans but also physical objects equipped with sensors/actuators, software and connectivity; communicate, collaborate, and act autonomously.

1.1.2 What is IoT?

The Internet of Things (IoT) describes a network of interconnected physical devices, or "things," that are equipped with sensors, software, and other technologies. These devices can collect, process, and exchange data with one another and with cloud-based systems. The goal is to create a seamless ecosystem where these objects can communicate and interact with minimal human intervention, enabling them to provide valuable, context-aware services.



Figure 2: The Internet of Things

The different types of "Things": There are many different types of IoT devices/objects, or "*Things*", ranging from simple sensors to more advanced devices. These devices, ranging from everyday objects like smartphones and thermostats to specialized equipment such as medical implants and agricultural sensors, are equipped with unique identifiers (IP addresses) and can communicate over networks.

But why now? Two forces made IoT feasible at scale: (i) cheap, low-power embedded hardware; (ii) pervasive connectivity (Wi-Fi, LPWAN, 4G/5G), supported by elastic cloud services and modern data/ML pipelines.

1.1.3 IoT Architecture and Components

The Internet of Things (IoT) ecosystem is commonly described through a layered reference architecture that highlights its key components and the flow of data from the physical world to user-facing applications. The main layers are:

1. **Perception Layer (Sensors and Devices):** This layer consists of devices equipped with sensors and actuators that capture real-world parameters such as temperature, humidity, vibration, or biometric signals. These devices act as the "eyes and ears" of IoT, bridging the physical and digital domains.
2. **Network Layer (Gateways and Connectivity):** To transmit collected data, devices rely on connectivity protocols such as Wi-Fi, Bluetooth Low Energy, Zigbee, LoRaWAN, or 5G. Gateways often serve as intermediaries, aggregating raw data and performing local preprocessing before forwarding information through IP-based routing.

3. **Data/Platform Layer (Cloud and Processing):** This layer provides scalable storage and computing resources. Gateways, brokers, and cloud platforms enable stream processing, device management, and data analytics. Using big data techniques, artificial intelligence, and machine learning, raw sensor readings are transformed into actionable insights.
4. **Application Layer (Applications and Services):** Insights are delivered to end users via dashboards, mobile applications, or automated control systems. These services support decision-making, enhance convenience, and in many cases provide autonomy to connected environments.

Together, these layers form an interdependent ecosystem. For example, a smart thermostat integrates sensors (temperature detection), connectivity (Wi-Fi), cloud-based optimization algorithms, and user-facing mobile applications to create an intelligent and energy-efficient household environment.

1.1.4 Industry Applications of IoT

IoT applications are now present across nearly every sector of society:

- **Smart Homes:** Connected lighting, thermostats, and security systems provide convenience, energy efficiency, and safety. Devices such as smart locks or voice assistants illustrate the growing consumer acceptance of ubiquitous connectivity.
- **Wearables:** Fitness trackers, smartwatches, and medical monitoring devices empower individuals to monitor health in real time, supporting personalized wellness and early detection of medical conditions.
- **Industrial IoT (IIoT):** Factories and energy plants employ IoT sensors to monitor equipment conditions, enable predictive maintenance, and optimize production lines. This results in significant reductions in downtime and operational costs.
- **Healthcare:** Remote patient monitoring through connected devices allows continuous observation outside hospital settings. Examples include glucose monitors for diabetes management, connected inhalers, or wearable ECG sensors.
- **Automotive:** Vehicles are increasingly equipped with IoT-enabled systems for navigation, diagnostics, and advanced driver assistance. Autonomous driving in particular depends on a dense network of sensors and real-time communication.

1.2 Overview of an IoT Ecosystem

1.2.1 Functional Building Blocks of IoT

A useful way to understand IoT systems is to consider their **functional building blocks**, which define the core operations of any deployment:

1. **Identification** — Each entity (device, user, or service) must be uniquely identifiable to ensure traceability and control.
2. **Sensing** — Sensors and actuators capture data from the environment and act upon it.
3. **Networking** — Communication technologies and protocols connect heterogeneous devices and systems, enabling data flow.
4. **Computation** — Processing occurs across a continuum: resource-constrained nodes, gateways, fog, and cloud infrastructures.
5. **Services** — Functionalities exposed to users or systems, such as querying temperature or controlling devices.
6. **Analytics** — Transformation of raw data into insights through statistical models, big data platforms, and AI algorithms.

Together, these building blocks capture the essential *what* of IoT systems—from sensing the physical world to delivering insights and control.

1.2.2 Non-functional Challenges of IoT Systems

Beyond functionality, IoT systems must also meet critical **non-functional requirements** that define the *quality* and *trustworthiness* of their operation:

- **Security:** Protecting the confidentiality, integrity, and availability of data and services.
- **Privacy:** Safeguarding personally identifiable information and usage data.
- **Interoperability:** Ensuring seamless collaboration across heterogeneous vendors, domains, and standards.
- **Scalability:** Supporting growth in devices, users, and services without performance degradation.

- **Latency:** Guaranteeing responsiveness to time-critical events, often through edge and fog computing.
- **Reliability:** Maintaining consistent operation despite failures or adverse conditions.

These challenges are prerequisites for sustainable and trustworthy IoT deployments.

1.2.3 Benefits and Risks of IoT

From an adoption perspective, the appeal of IoT lies in the balance between its potential benefits and associated risks, particularly at the societal and business levels.

Benefits:

- **Efficiency and Optimization:** Automated systems reduce waste, streamline workflows, and conserve resources.
- **Convenience:** Everyday tasks are simplified through voice commands, remote control, and predictive automation.
- **Cost Reduction:** Predictive maintenance lowers industrial repair costs, while energy-aware devices reduce household bills.
- **Innovation and Business Opportunities:** Data-driven services enable new business models, from smart cities to usage-based insurance.

Risks:

- **Surveillance and Loss of Autonomy:** The pervasiveness of IoT devices raises ethical concerns about monitoring and control.
- **Dependence and Vulnerability:** Over-reliance on connected devices can disrupt daily life in case of outages or attacks.
- **Economic and Social Impacts:** Widespread automation may displace jobs or reshape traditional industries.
- **Ethical Concerns:** Questions of data ownership, consent, and accountability remain unresolved.

The dual nature of IoT—as both an enabler of progress and a potential source of societal risk—underscores the need to embed trustworthiness at every stage of its design and deployment.

1.2.4 The computational layers in IoT systems

The location where computation takes place can vary along a continuum, from the smallest sensor to large-scale data centers. The main paradigms are:

1. **Mist Computing:** Computation performed directly on microcontrollers or embedded sensors. Extremely close to the data source, offering ultra-low latency but with very limited processing power.
Dew Computing: A specialized form of edge computing where computation happens on personal devices (e.g., PCs, smartphones) that may operate offline and later synchronize with the cloud.
Computation: $\rightarrow$ on or inside the sensor/thing (microcontroller); ultra-low latency, highly constrained.
2. **Edge Computing:** Computation carried out on devices located near the sensors, such as gateways, routers, or specialized edge servers. Provides low latency and reduces bandwidth consumption by processing data locally before sending it onward.
Computation: $\rightarrow$ on-device or nearby gateway; fast reactions, local aggregation/ML.
3. **Fog Computing:** An intermediate layer between edge devices and the cloud, typically involving local or regional servers. Fog nodes aggregate and preprocess data from multiple devices, enabling real-time analytics closer to the source.
Computation: $\rightarrow$ local/intermediate servers (plant/floor/metro level) aggregating multiple edges; pre-processing, orchestration.
4. **Cloud Computing:** Centralized computation and storage in large-scale data centers, often geographically distant. Offers virtually unlimited resources and scalability but comes with latency, bandwidth, and privacy trade-offs.
Computation: $\rightarrow$ centralized data centers; large-scale storage, analytics, model training, fleet management.
5. **On-Premises (Local) Computing:** Traditional computing infrastructure maintained directly on site by organizations, ensuring control, compliance, and privacy at the cost of scalability.

Together, these paradigms form a continuum: *Mist* $\rightarrow$ *Edge* $\rightarrow$ *Fog* $\rightarrow$ *Cloud*, with On-Premises and Dew computing providing alternative models depending on context and requirements.

	Mist	Edge	Fog	Cloud
Typical location	Sensor / MCU	Device / gateway	Local servers (site / metro)	Remote data centers
Latency	Very low	Low	Moderate	Higher
Compute & storage	Minimal	Limited–moderate	Moderate–high	Very high
Bandwidth use	Minimal (pre-filtered)	Reduced	Reduced	Highest (if raw data)
Management	Hard (heterogeneous)	Harder at scale	Managed via site ops	Centralized, mature
Primary use	Control loops	Immediate analytics	Aggregation & orchestration	Big data & model training

Table 1: Comparison of different computation layers (at a glance)

1.3 Threats in the Era of Machine-to-Machine Communication

The evolution from the Internet of Content, Services, and People toward the Internet of Things (IoT) introduces a new paradigm: *machine-to-machine* (M2M) communication. Unlike earlier internet stages, where humans were central to interaction, IoT ecosystems enable autonomous exchanges among billions of devices. This transformation amplifies existing threats and introduces new ones, reshaping the cybersecurity landscape.

1. **Attack Surface Expansion:** Each connected object represents a potential entry point. Billions of heterogeneous IoT devices dramatically increase the attack surface compared to conventional IT systems, which involve standardized servers and workstations.
2. **Resource-Constrained Devices:** Many IoT nodes operate with minimal power, memory, and processing capabilities. Unlike traditional IT endpoints, these limitations prevent the deployment of strong encryption, complex authentication, or continuous monitoring, making devices easier to compromise.
3. **Heterogeneity and Interoperability Challenges:** The diversity of vendors, hardware, firmware, and communication protocols complicates

uniform security enforcement. This contrasts with IT systems, where standards and uniform platforms reduce variability and security gaps.

4. **Longevity of Devices:** IoT deployments often span 5–15 years or more, yet many devices receive little or no maintenance or software updates. Traditional IT hardware usually has shorter lifecycles with regular patching, reducing long-term vulnerabilities.
5. **Privacy and Data Sensitivity:** IoT devices continuously collect environmental, health, and geolocation data. Breaches can compromise personal privacy, enable persistent tracking, or erode trust, going beyond the mostly financial or reputational impact typical of traditional IT breaches.
6. **Physical Safety Risks:** IoT attacks can directly affect the physical world, e.g., hijacking vehicles, disabling medical devices, or sabotaging industrial systems. Traditional IT threats rarely have direct safety consequences.
7. **Manipulation of Data and Machine Learning Systems:** Automation and predictive analytics rely heavily on sensor data. Adversaries may poison or tamper with these streams, leading to unsafe decisions or system failures—a risk largely absent in conventional IT environments.

In summary, M2M communication not only multiplies the scale of traditional threats but also extends their consequences into the physical domain. Securing IoT requires novel frameworks, standards, and approaches that balance protection with continued innovation, ensuring trust and safety in increasingly interconnected environments.

1.4 Activity

Given a small IoT scenario (e.g., smart greenhouse), map: devices, data paths, computing placement, and security controls. Justify each choice with latency, bandwidth, and risk arguments.

2 Introduction to Cybersecurity

Discussion : Do people really need security in exchanging information? Think about these scenarios:

- You are designing a new product, you might be solely working on it on your computer or exchanging information and communicating with others via internet
- You have a private conversation with someone. How to feel secure that this conversation cannot be read by someone?
- National security data regarding criminals must be digitally saved somewhere.
- You have an online store. How can you ensure that people are operating within their rights?

2.1 Cybersecurity vs Computer Security

Cybersecurity is the discipline that focuses on protecting systems, networks, and data across all digital environments, from personal devices to global IT infrastructures. It addresses threats such as hacking, phishing, ransomware, and DDoS attacks, aiming to secure data and services against sophisticated adversaries.

Computer Security focuses specifically on safeguarding individual computers and the information they process. Its goals are to maintain the **confidentiality, integrity, and availability (CIA)** of data on a personal device or small network. Tools include antivirus software, firewalls, and encryption. While overlapping with cybersecurity, computer security operates on a smaller, more focused scale.

Information Systems are the backbone of both fields. An information system collects data from multiple sources, processes it, and provides meaningful output. Its resources include: {Hardware, Software and firmware¹, Data and information², Telecommunications and network infrastructure}.

¹Firmware is specialized software embedded into hardware, providing low-level control for device functionality.

²Data are raw facts; information is processed data with context and meaning.

2.2 Core Security Concepts: A Chain of Events

At the heart of cybersecurity is the protection of assets—the valuable hardware, software, data, and networks in a system. These assets are constantly at risk from a threat agent, which is any entity, like a hacker or eavesdropper, that poses a potential attack. A threat is the possibility of harm, which can be realized if a vulnerability—a weakness in the asset’s security—is found and exploited. The action of exploiting this weakness is called an attack.

To prevent attacks, we use security measures like encryption, a process that makes information secret and unreadable. Identification is another key defense, ensuring that people and objects are who they claim to be. This is done through unique identifiers like ID cards, secrets like passwords, and biometric characteristics such as fingerprints.

2.3 Challenges in Security

Cybersecurity is a constant challenge because it’s an unequal fight. A defender must secure all vulnerabilities, while an attacker only needs to find and exploit one. Systems are inherently complex, with multiple components and layers, making them difficult to secure completely. Furthermore, the benefits of security are often invisible until a system fails, which makes it hard to justify the continuous monitoring and updates required to keep it safe. Ultimately, no single security mechanism is perfectly efficient, making a multi-layered defense essential.

2.4 The CIA triad

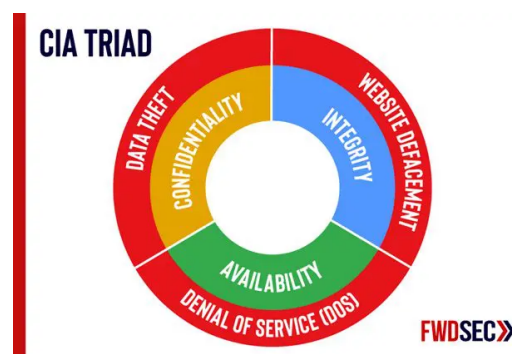


Figure 3: Security CIA triad

The CIA triad— **Confidentiality, Integrity, and Availability** —is a guiding model in information security. A comprehensive information security

strategy includes policies and security controls that minimize threats to these three crucial components. Cybersecurity involves protecting data and information and software from unauthorized access, deletion or modification And to provide confidentiality, integrity and availability.

Confidentiality involves preventing any access of data to unauthorized individuals. It ascertains the identity of authorized personnel involved in sharing and holding data secure, private and anonymous. Confidentiality can be compromised by hackers who crack poorly encrypted data, incorporate various types of cyber attacks, and disclose sensitive data.

Integrity is the condition of protecting the data/information from being altered by unauthorized individuals. integrity can be compromised, especially by cyber-crimes when malware is embedded into webpages or when a machine/object is turned into a "zombie computer" ³

Availability is ensuring that authorized personnel has access to the data or information when needed. Unavailability of information usually occurs when security incidents such as human error, programming errors, DDoS attacks, or hardware failures take place.

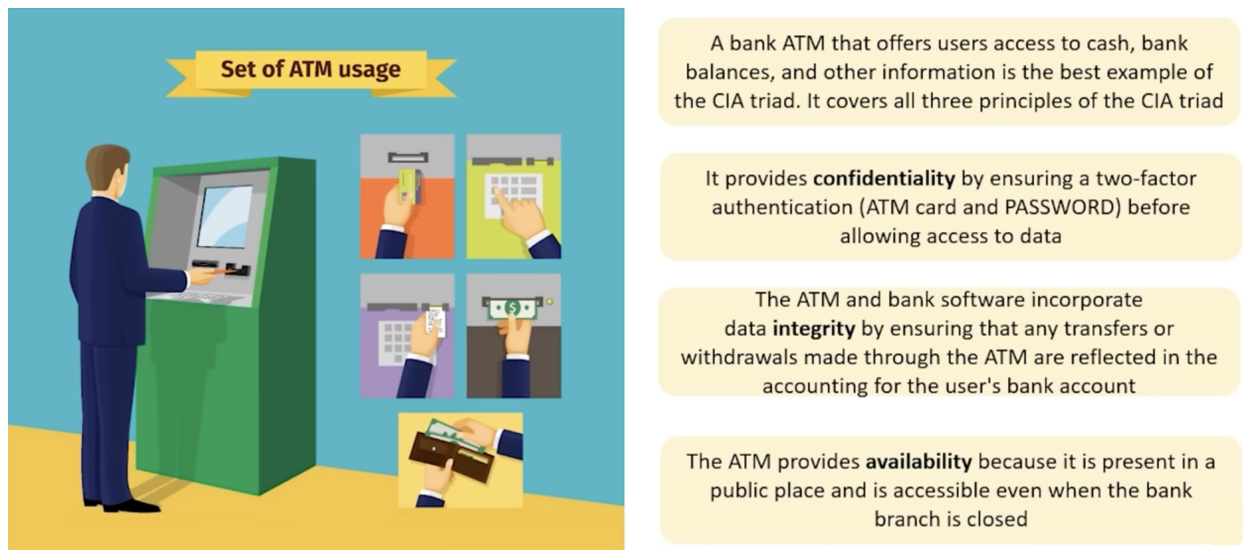


Figure 4: Money withdrawal from an ATM - Example of a CIA triad

³In computing, a zombie is a computer connected to the Internet that has been compromised by a hacker via a computer virus, computer worm, or Trojan horse program and can be used to perform malicious tasks under the remote direction of the hacker.

2.5 Layers of Security Vulnerabilities

2.5.1 Hardware Vulnerabilities:

Hardware vulnerabilities arise when physical devices are manipulated or compromised in ways that degrade their security or functionality. This can involve adding, modifying, or removing devices, intercepting communication between devices, or overwhelming devices with excessive traffic (also known as a Denial of Service (DoS) attack). These vulnerabilities can be exploited to cause system failure, data theft, or unauthorized access.

Example: An example of a hardware vulnerability is a malicious keyboard. If infected with malware, a keyboard could be used to log and capture all of your keystrokes (a technique known as keylogging), allowing an attacker to steal sensitive information such as passwords or financial data. Additionally, a malicious keyboard might stop functioning entirely, rendering the device useless and disrupting user activity.

2.5.2 Software Vulnerabilities:

Software vulnerabilities occur when flaws or weaknesses within the software are exploited, either by modifying, corrupting, or destroying code, or by replacing legitimate software with malicious versions. These attacks can lead to unauthorized access, data loss, or control over the system. These vulnerabilities often arise due to coding errors, lack of proper security controls, or misconfigurations, and can be exploited intentionally by attackers or accidentally triggered.

Example: A common example of software vulnerability is a buffer overflow attack. This occurs when an application improperly handles data input, allowing an attacker to overwrite parts of the memory, leading to arbitrary code execution or a crash. This can allow attackers to execute malicious code and take control of the system.

2.5.3 Data Vulnerabilities:

Data vulnerabilities stem from the fact that data is valuable and can be a target for theft, manipulation, or destruction. The sensitivity and value of data may vary, but any unauthorized access or alteration can have severe consequences. Data vulnerabilities can arise from poor encryption practices, inadequate access controls, or improper data storage and transmission.

Example: A common example of a data vulnerability is a ransomware attack. In such an attack, an attacker encrypts a victim's data and demands payment in exchange for the decryption key. This not only threatens the

integrity and availability of the data but can also result in significant financial and reputational damage

2.5.4 Network Vulnerabilities:

Network vulnerabilities occur when weaknesses in network design, configuration, or implementation allow attackers to infiltrate or disrupt the communication between devices. These vulnerabilities can lead to unauthorized access, data theft, eavesdropping, or even full control of networked systems. Network vulnerabilities are often exploited through techniques such as sniffing, spoofing, and Distributed Denial of Service (DDoS) attacks. Poorly configured firewalls, unpatched network protocols, or insecure communication channels can all contribute to these security gaps, making networks a prime target for cyberattacks.

Example: A common example of a network vulnerability is a man-in-the-middle (MITM) attack. In this attack, an attacker secretly intercepts and potentially alters the communication between two parties who believe they are directly communicating with each other. This allows the attacker to steal sensitive information, such as login credentials or financial data, or to manipulate the transmitted data for malicious purposes.

2.6 Elements of Cybersecurity

Cybersecurity encompasses a broad range of practices and technologies designed to protect systems, networks, and data from malicious attacks. Below are some of the key elements that form the foundation of a comprehensive cybersecurity strategy:

1. **Application security** focuses on protecting software applications from vulnerabilities and threats that could compromise their functionality, data integrity, or user privacy. This includes securing the development, deployment, and ongoing maintenance of applications by implementing measures such as encryption, authentication, and regular updates to patch security flaws.
2. **Information security** involves safeguarding sensitive **data** from unauthorized access, disclosure, modification, or destruction. It encompasses the protection of data in both digital and physical forms, ensuring confidentiality, integrity, and availability of information across systems, whether stored or in transit.
3. **Network security** is concerned with protecting the infrastructure of a computer network from attacks, misuse, and unauthorized access. This

includes securing network devices, preventing breaches, and implementing measures like firewalls, intrusion detection systems, and encryption to maintain a safe communication environment. The most well-known network security parts are as per following: Firewalls, Antivirus, Email Security, Web Security, Wireless Security.

4. **Disaster recovery planning** involves preparing an organization for unexpected incidents like cyber-attacks, natural disasters, or system failures that could disrupt operations. This element focuses on developing strategies and procedures to restore systems, data, and functionality as quickly as possible, ensuring business continuity and minimizing downtime.
5. **Operational security** is the practice of identifying and protecting critical information and procedures that could be exploited by adversaries. This includes controlling and monitoring the handling of data, implementing access controls, and maintaining secure operational processes to prevent leaks and unauthorized access.
6. **End-user security** focuses on protecting individuals who interact with systems and networks from cyber threats. This element addresses the human factor in cybersecurity by promoting awareness, education, and best practices, such as using strong passwords, recognizing phishing attempts, and maintaining up-to-date security software on personal devices.

2.7 Security Laws

A range of laws and regulations govern the handling and protection of various types of data, particularly personal and sensitive information. These laws impose a legal obligation on data custodians to ensure that personal information is securely managed and protected. Additionally, many trade agreements and legislation restrict the unauthorized replication and dissemination of proprietary data.

Other laws emphasize safeguarding individuals' privacy by mandating the protection of personal information both in publicly available datasets and in insights derived from data analytics.

Trust in data security largely stems from the robustness and reliability of the technology used to ensure the confidentiality and integrity of information, protecting it from malicious actors and potential breaches.

2.8 Discussions

- What do you think about authentication mechanisms based on IP addresses? example you can access a university library's website when you are in a precise geographical zone. In a range very close to the library

3 Cyberattacks

Discussions

- Imagine you find a travel luggage with a 3 digit password lock. What would you do to unlock it?
- Imagine you find a travel luggage with a 3 digit password lock and you suspect or know the identity of its real owner. What would you do to unlock it?
- Search for "Address Resolution Protocol" attack and explain and classify it.
- Think about a malicious person who knows your social security number, your date of birth , your credit card and your address. What scenarios can you imagine? Being a victim of personal data theft will make you feel helpless. But there is ways to keep your data safe and confidential and this known as cybersecurity. Cybersecurity protects your personal data, your devices and your assets such as your bank account and your photos.

How do cyber attacks start? Cyberattacks can start in several ways, depending on the attack's nature and objectives. Attackers identify and exploit security flaws in software, hardware, or network systems, and even in human nature. These attacks can be sophisticated and multi-layered, often combining several techniques to achieve their goals. Effective cybersecurity involves using a combination of technical defenses, user education, and vigilance to protect against these varied threats. Attacks in security can be classified into several categories based on the method, target, or nature of the attack. Below is a classification of some of the most known security attacks.

3.1 Passive attacks

Passive attacks aim to gather information without altering the system. These attacks typically focus on eavesdropping on or monitoring data transmissions. Goal: Data collection without detection.

3.1.1 Eavesdropping attack

Eavesdropping: An eavesdropping attack refers to the unauthorized real-time interception of private communication, aiming to eavesdrop on the

information transmission process and potentially leak privacy. Usually this is easy by intercepting unencrypted network traffic.

3.1.2 Traffic Analysis

Observing the pattern of communication, such as who is communicating with whom, without necessarily accessing the content of the messages.

3.2 Active Attacks

Definition: Active attacks involve modification of the data or the creation of fraudulent data. These attacks aim to alter, disrupt, or destroy data or systems. Goal: Data alteration, disruption, or destruction.

3.2.1 Man-in-the-Middle Attacks:

Attackers intercept and potentially alter communications between two parties without their knowledge. This can be done through techniques like session hijacking or eavesdropping on unsecured networks.

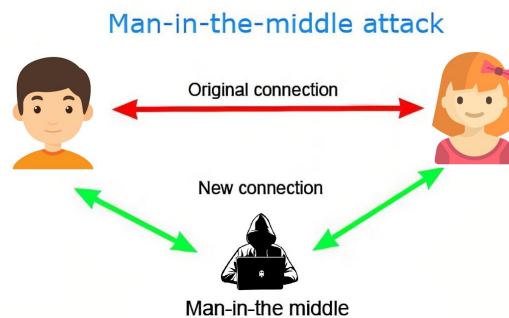


Figure 5: Man-in-The-Middle attack

3.2.2 SQL injection

SQL injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database.

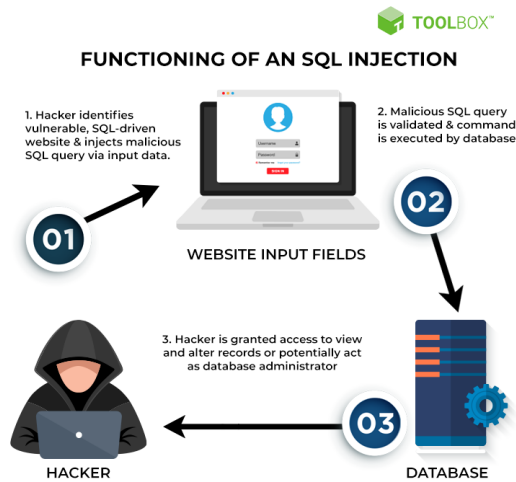


Figure 6: SQL injection

3.3 Insider Attacks

Definition: Insider attacks are carried out by individuals who have authorized access to the system but misuse their privileges. These attackers may be employees, contractors, or partners. **Examples:** Data Theft: Stealing sensitive data from within the organization. Privilege Abuse: Using access rights to make unauthorized changes or bypass security controls. Sabotage: Deliberately destroying data or causing harm to systems. **Goal:** Malicious or accidental misuse of authorized access.

3.4 Outsider Attacks

Definition: Outsider attacks are initiated by individuals who do not have authorized access to the system. These attackers breach security measures from outside the organization. **Goal:** Gaining unauthorized access to a system or data. **Examples:**

Brute Force Attacks: Systematically trying different passwords until the correct one is found.

Phishing: Deceiving individuals into providing sensitive information (e.g., passwords, financial information).

Exploiting Software Vulnerabilities: Using weaknesses in software to gain unauthorized access.

3.5 Distributed Attacks

Distributed attacks involve multiple systems working together to carry out an attack. They typically aim to overwhelm or disrupt services. A distributed denial-of-service (DDoS) attack is a malicious attempt to disrupt the normal traffic of a targeted server, service or network by overwhelming the target or its surrounding infrastructure with a flood of Internet traffic., causing it to become slow or unresponsive. This is often done using a network of compromised devices (a botnet).

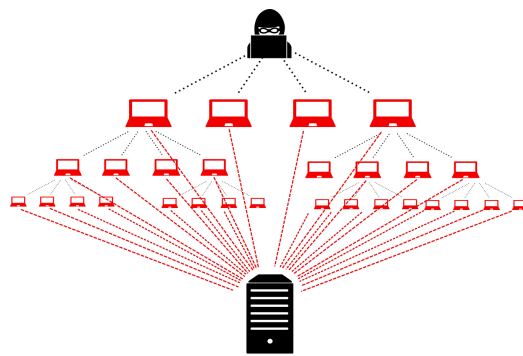


Figure 7: DDoS attacks

Goal: Overwhelm a system or network using numerous attacking sources.

3.6 Social Engineering Attacks

Social engineering attacks involve manipulating individuals into divulging confidential information or performing actions that compromise security. Phishing is a subset of social engineering.

→ Goal: Trick users into compromising security. Attackers manipulate individuals into divulging confidential information or performing actions that compromise security. This can include impersonating trusted figures or creating a sense of urgency.

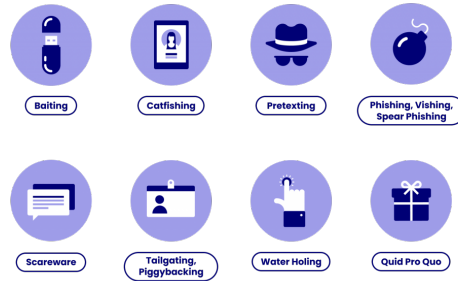


Figure 8: Most common social engineering attacks

- **Insider Threats:** Disgruntled or malicious employees or contractors intentionally compromise security by leaking information or causing other types of damage.
- **Phishing:** Attackers send deceptive emails or messages to trick individuals into revealing sensitive information, such as login credentials. These messages often appear to be from legitimate sources.



Figure 9: Phishing attack

- **Scareware** is a type of malware tactic used to manipulate victims into downloading or buying potentially malware-infested software. Scareware attacks are used by scammers and cybercriminals to trick users into thinking they have to buy/download malware disguised as real cybersecurity software.



Figure 10: Scareware malicious pop up

- **Baiting:** is a type of social engineering attack where a scammer uses a false promise to lure a victim into a trap which may steal personal and financial information or inflict the system with malware. The trap could be in the form of a malicious attachment with an enticing name.



Figure 11: Baiting a victim

- **Quid pro quo:** a favour or advantage granted in return for something.

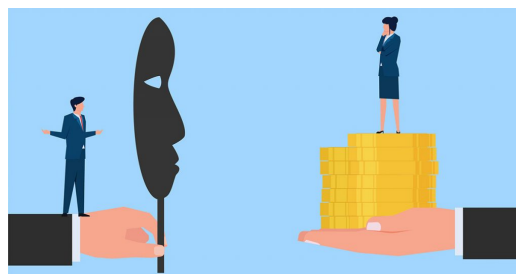


Figure 12: Quid Pro Quo (a type of bribe)

3.7 Malware Attacks

Malware (malicious software) refers to programs designed to damage, disrupt, or gain unauthorized access to computer systems. Goal: Cause harm, steal data, or extort victims.

3.7.1 Viruses

A computer virus attaches itself to a program or file enabling it to spread from one computer to another, leaving "infections" as it travels. Once activated, they replicate by infecting other files or programs, often causing damage such as corrupting data, slowing down system performance, or crashing applications. Viruses can be spread through email attachments, file downloads, or even compromised websites, and they usually need some form of user interaction to propagate further. They come in a range of severity, some may cause mildly annoying effects while others can damage hardware, software or files.

3.7.2 Worms

worms, in contrast, are self-replicating programs that do not need human interaction to spread. They exploit vulnerabilities in networks and systems to propagate autonomously, moving from one device to another. Worms can spread rapidly across networks, consuming resources like memory and network bandwidth, causing system slowdowns, sometimes leading to data breaches or complete system failures. Their ability to spread independently makes them particularly dangerous, as they can cause widespread disruption in a very short amount of time.

3.7.3 Trojan Horse

Trojans are malicious software programs designed to appear as legitimate or harmless applications. They deceive users into installing them, often by masquerading as useful software, while secretly performing harmful actions like stealing data, deleting files and destroying information. Trojans compromise security by allowing unauthorized access to the system. Unlike viruses or worms, Trojans do not self-replicate but rely on social engineering tactics to infiltrate systems under the guise of trusted software.

Malicious software, such as viruses, ransomware, or spyware, is delivered through various means (e.g., infected email attachments, compromised websites). Once installed, malware can steal data, disrupt operations, or cause other harm.

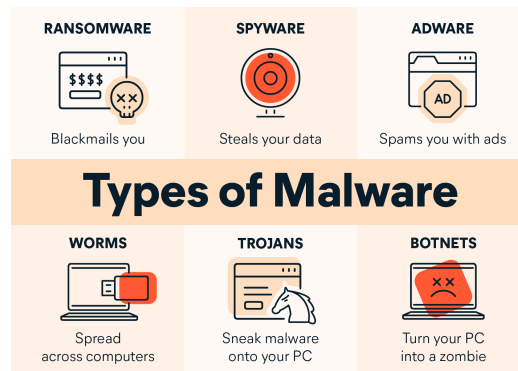


Figure 13: Types of Malware

3.8 Physical Attacks

Physical attacks target the actual hardware or infrastructure supporting a system. This could involve physically tampering with devices, stealing hardware, or damaging critical infrastructure. Goal: Physically compromise devices or infrastructure.

Examples:

Device Theft: Stealing laptops, smartphones, or other hardware that contains sensitive information.

Tampering: Physically altering or sabotaging hardware to disrupt functionality or extract data.

Side-Channel Attacks: Extracting information from the physical implementation of a system (e.g., analyzing electromagnetic emissions).

3.9 Password cracking Attacks

3.9.1 Brute Force Attacks

Automated tools are used to guess passwords or encryption keys by trying numerous combinations rapidly until the correct one is found.

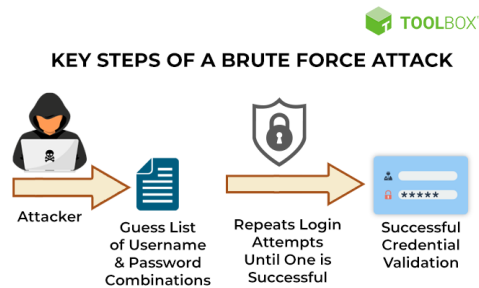


Figure 14: Brute force attack

3.9.2 Credential Stuffing

Credential stuffing is a type of cyberattack where attackers use automated tools to try a large number of stolen username-password pairs on various websites or services. The goal is to gain unauthorized access to accounts. This attack exploits the fact that many people reuse the same credentials across multiple sites.

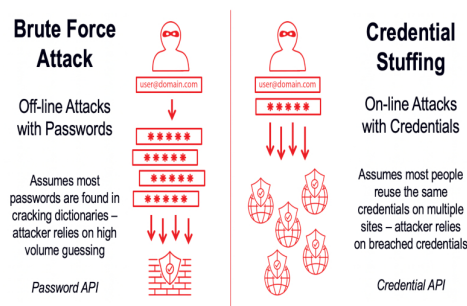


Figure 15: Credential stuffing

3.10 Advanced Persistent Threats (APTs)

Definition: APTs are prolonged and targeted attacks where an attacker gains unauthorized access to a network and remains undetected for an extended period. **Examples:** State-Sponsored Espionage: Nation-state actors targeting government or corporate networks to steal sensitive data. Corporate Espionage: Competitors infiltrating a network to steal trade secrets. **Goal:** Long-term infiltration and data theft without detection.

4 Network security and attacks

4.1 Elements of a network

A comprehensive understanding of network elements is essential before diving into the various types of network attacks. Networks are complex systems with interconnected devices and protocols, each playing a crucial role in data communication and security. Below are the key components that form the foundation of a network and are frequently targeted or exploited in cyberattacks.

4.1.1 Network Architecture

Networks can be classified into different types, such as Local Area Networks (LAN), which operate within a small geographical area (e.g., a home or office), and Wide Area Networks (WAN), which connect larger, more dispersed areas (e.g., the internet). Wireless networks, such as WiFi, allow devices to connect without physical cables. Additionally, networks are arranged in various topologies like star, mesh, and bus, which describe how devices are physically or logically interconnected. Understanding these types and topologies is vital because they influence the flow of data and potential points where attacks can occur.

4.1.2 Devices in a network

Devices form a network and facilitate communication and data transfer within it. **Servers** provide data and services to clients, which request them. Servers are common targets for attacks, such as denial-of-service (DoS) attacks, while clients can be exploited through malware or phishing schemes. **Routers** direct data between networks, ensuring that data packets reach their intended destination, and switches enable communication within a network. These devices are critical in maintaining efficient and secure data flows, making them prime targets for attacks aimed at redirecting or intercepting traffic. **Firewalls**, on the other hand, act as security barriers, filtering network traffic based on defined rules, and play a key role in blocking unauthorized access.

4.1.3 Protocols

At the core of internet communication lies the TCP/IP model, which consists of four layers: Link, Internet, Transport, and Application. These layers work together to ensure that data is transmitted between devices. For example, the Internet layer handles IP addressing and routing, while the Transport

layer ensures reliable data transfer. Attackers often exploit weaknesses in these layers through attacks like IP spoofing or TCP hijacking. Common network protocols, such as HTTP/HTTPS for web traffic, DNS for domain name resolution, and SMTP for email, are vital for communication but are also frequent targets for attacks like man-in-the-middle, DNS poisoning, and email spoofing.

4.1.4 IP Addresses and DNS

IP addresses serve as unique identifiers for devices on a network, enabling them to communicate with each other. There are two versions: IPv4, and IPv6, which was developed to accommodate the growing number of devices. Attackers often exploit IP addresses through tactics like IP spoofing, where they disguise malicious traffic as legitimate. Domain Name System (DNS) is another key element, responsible for translating human-readable domain names (e.g., `www.example.com`) into IP addresses. Attackers frequently target DNS to redirect traffic or launch attacks like DNS hijacking and cache poisoning.

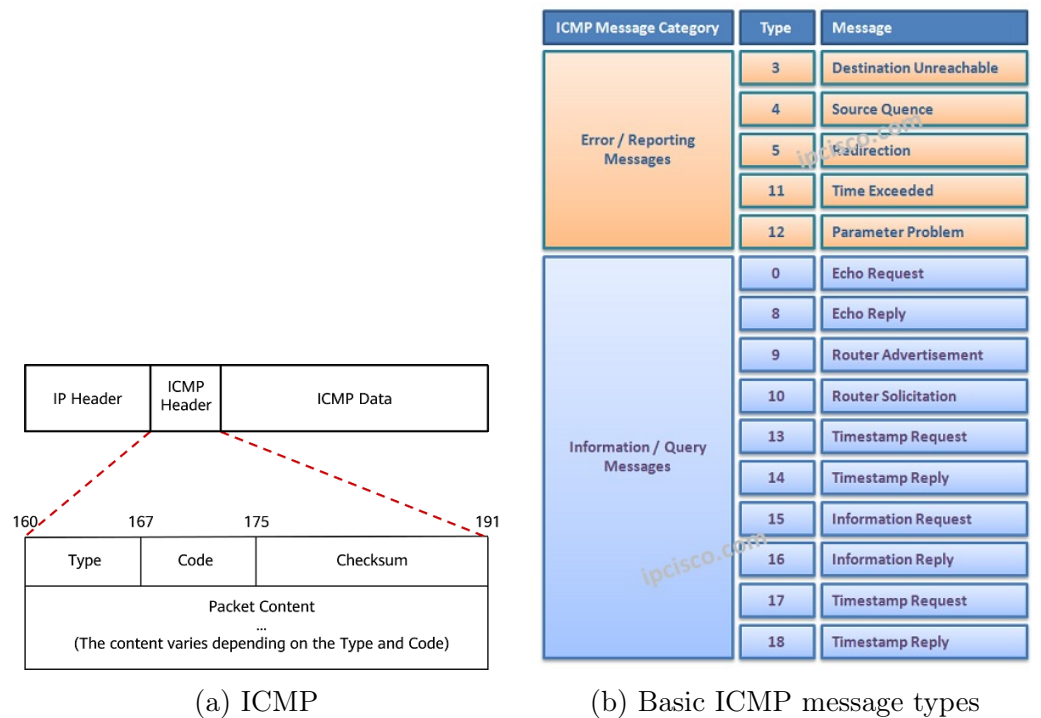
4.1.5 Network Traffic

Data within a network is transmitted in small units called packets, which contain source and destination IP addresses, as well as the actual data being sent. Attackers can intercept, manipulate, or block these packets through techniques like packet sniffing or man-in-the-middle attacks. Additionally, the concept of bandwidth, or the maximum rate of data transfer, is crucial in understanding network performance. Attacks like Distributed Denial of Service (DDoS) attempt to overwhelm a network's bandwidth, rendering it unusable for legitimate users.

4.1.6 Internet Control Message Protocol (ICMP)

The Internet Control Message Protocol (ICMP) is a crucial network layer protocol used to transmit control messages between hosts and routers to report whether hosts are reachable and routes are available; **it is an error handling and debugging protocol**. It plays an important role in the TCP/IP protocol suite and is typically used by the IP (or higher layer protocols as TCP or UDP), it is encapsulated inside an IP header. Although these control messages do not transmit user data, they are crucial for collecting network information, diagnosing and rectifying network faults, and transmitting user data.. Although ICMP is vital for network troubleshooting, it can also be exploited in certain types of network attacks, such as ICMP "flood attacks"

or "ping of death", where attackers overload a target system with excessive ICMP requests (or ping request) to disrupt network services.



that monitors and controls incoming and outgoing network traffic based on predefined security rules. It acts as a barrier between a trusted internal network and untrusted external networks, such as the internet. Firewalls can be implemented as hardware, software, or a combination of both. They filter traffic to prevent unauthorized access, block malicious traffic, and allow legitimate communications to pass through. Effective firewall policies help mitigate risks such as unauthorized access, denial-of-service (DoS) attacks, and data exfiltration.

4.2.2 Password policies

Password policies are critical for protecting sensitive systems and data from unauthorized access. These policies define the rules for creating, managing, and using passwords within an organization. Typically, they include requirements for password complexity, such as minimum length, use of special characters, and a combination of uppercase and lowercase letters. Additionally, password policies may enforce password expiration, requiring users to change their passwords regularly, and mandate multi-factor authentication (MFA) for added security. A strong password policy reduces the risk of successful brute force attacks, credential theft, and unauthorized access to sensitive systems.

4.2.3 Intrusion detection systems

Intrusion detection systems (IDS) are a crucial part of network security policies, designed to detect and respond to unauthorized or malicious activity on a network. An IDS monitors network traffic, looking for signs of security breaches, abnormal behavior, or policy violations. It operates in real-time, analyzing network traffic patterns to identify potential threats such as unauthorized access, hacking attempts, malware, or insider threats. Once a potential intrusion is detected, the IDS generates alerts for administrators to take corrective action. Network security policies must define how the IDS is configured, which types of intrusions should trigger alerts, and what responses should be taken to mitigate threats.

4.2.4 Anti-malware tools

Anti-malware tools are software solutions that protect networks and systems from malicious software, such as viruses, worms, Trojans, spyware, and ransomware. These tools are designed to detect, prevent, and remove malware before it can cause harm to the network. Anti-malware tools use signature-based detection, heuristic analysis, and behavioral monitoring to identify known and emerging threats. Network security policies should specify how

anti-malware tools are deployed, maintained, and updated to ensure that they remain effective against evolving threats. In addition, policies should address how frequently systems are scanned and how incidents of malware infection are handled to prevent widespread damage within the network.

4.2.5 Virtual Private Networks (VPNs)

Virtual Private Networks (VPNs) are an integral part of network security policies, particularly for organizations that have remote workers or need to secure communications over public networks. A VPN creates a secure, encrypted connection (often referred to as a "tunnel") between the user's device and the organization's network, ensuring that data transmitted over the internet remains confidential and protected from eavesdropping or interception. VPNs are essential for safeguarding sensitive information, such as corporate data, login credentials, and personal information, especially when accessing the network from untrusted locations like public Wi-Fi. Network security policies should define when and how VPNs should be used, the type of encryption required, and the authentication mechanisms needed to establish a VPN connection. This ensures that all remote communications are secure and that the network remains protected from external threats.

4.2.6 Authentication Policies

Explanation: Authentication policies define the methods and standards for verifying the identity of users, devices, or systems attempting to access the network. Effective authentication ensures that only authorized personnel can interact with sensitive data, systems, and resources. Authentication policies may include a variety of techniques, such as password-based authentication, multi-factor authentication (MFA), biometrics, and certificate-based authentication.

- **Password-Based Authentication:** The policy should outline the requirements for creating and maintaining secure passwords. This includes enforcing complexity (minimum length, special characters, mixed case), password expiration, and regular changes to minimize the risk of password compromise. Additionally, password storage practices, such as the use of hashing algorithms, should be specified to prevent unauthorized access to password databases.
- **Multi-Factor Authentication (MFA):** MFA enhances security by requiring users to provide two or more verification factors (e.g., a password combined with a code sent to a mobile device or a fingerprint scan). This reduces the risk of compromised credentials being used to gain

unauthorized access. The policy should mandate MFA for critical systems, remote access, and sensitive areas of the network.

- **Biometric Authentication:** Depending on the level of security required, the policy may include biometric authentication (e.g., fingerprint, facial recognition) as an additional layer of protection. Biometric data must be securely stored and managed to ensure privacy and security.
- **Certificate-Based Authentication:** This policy can also enforce the use of digital certificates to authenticate devices or users on the network. Public Key Infrastructure (PKI) can be used to issue and manage certificates that verify the identity of users and devices, allowing them to securely access network resources.
- **Single Sign-On (SSO):** For convenience and security, authentication policies can also implement Single Sign-On (SSO), which allows users to access multiple systems with a single set of credentials. While SSO enhances usability, the policy should ensure that strong authentication is used to protect access to all connected systems.
- **Authentication Failures and Lockout:** The policy should define procedures for handling failed authentication attempts, including lockout mechanisms to prevent brute force attacks, and processes for securely restoring access when legitimate users are locked out.

By enforcing strong authentication mechanisms, organizations can significantly reduce the likelihood of unauthorized access, insider threats, and credential-based attacks, such as phishing or social engineering.

5 History of Cryptographic tools

The history of decryption is a rich and fascinating journey, deeply intertwined with cryptography. It spans centuries and reflects both human ingenuity and the ongoing battle between secrecy and exposure. Here's an overview:

5.1 Ancient Encryption and Decryption:

Egyptians (c. 1900 BCE): One of the earliest known uses of encryption comes from ancient Egypt, where non-standard hieroglyphs were used to obscure important writings. This was more about hiding meaning from the average person rather than a cryptographic method requiring decryption.

5.2 Spartan Scytale (c. 400 BCE):

Ancient Greeks, particularly Spartans, used a device called a scytale for encryption. It involved wrapping a strip of parchment around a rod and writing across it. When unwrapped, the text appeared jumbled unless the recipient had a rod of the same size to rewrap the parchment and reveal the message.

5.3 Caesar Cipher (c. 58–50 BCE):

Julius Caesar famously used a substitution cipher to communicate with his generals during wartime. The cipher shifted letters by a fixed number of positions in the alphabet, requiring the recipient to reverse the shift (i.e., decrypt) to read the message.

Vulnerabilities: Susceptible to **brute force attacks** because of the small number of possible shifts (only 25 options in the English alphabet). It also lacked defense against **frequency analysis**, as letter patterns were preserved.

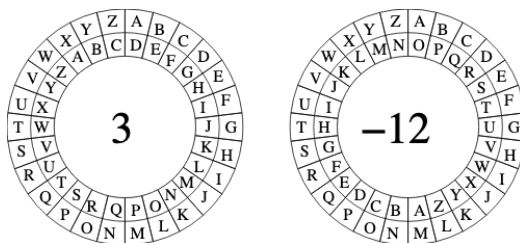


Figure 17: Examples of 2 different possible shifts

5.4 Middle Ages and Renaissance (500–1500 CE)

Arabic Cryptography (9th Century): In the Arab world, significant advances were made in cryptanalysis. Al-Kindi, an Arab mathematician and polymath, wrote a treatise called "A Manuscript on Deciphering Cryptographic Messages," which introduced frequency analysis. This method made it possible to break substitution ciphers by analyzing the frequency of letters in the ciphertext.

5.5 Vigenère Cipher (16th Century)

Developed by Blaise de Vigenère, this cipher used a series of different Caesar ciphers based on a keyword, making it more resistant to frequency analysis. Though it was not broken until the 19th century, it represented an important step in encryption.

Fixes: The use of a keyword increases the complexity and mitigates frequency analysis attacks.

Vulnerabilities: Still breakable with *Kasiski examination*⁴ and *frequency analysis*, especially when the keyword is short or reused.

5.6 Early Modern Period (1500–1800)

Cryptography in Diplomacy and War: During the Renaissance and early modern periods, European states began to use increasingly complex ciphers in diplomacy and warfare. Encryption became a key tool in statecraft, used by figures like Mary, Queen of Scots, whose encrypted correspondence was decrypted, leading to her execution for treason ⁵.

Fixes:

- Enhanced Obfuscation: Compared to simpler techniques like Caesar ciphers or the scytale, Mary's substitution cipher made messages harder to break by random guessing. It replaced letters with symbols or numbers, effectively hiding the structure of the original message.
- Defeats Frequency Analysis (Partially): By using symbols or complex substitutions, the cipher made it more challenging (though not impossible) to apply traditional frequency analysis, which was effective against simple ciphers like Caesar.

⁴https://en.wikipedia.org/wiki/Kasiski_examination

⁵Mary, Queen of Scots, used a substitution cipher to encrypt her correspondence during her imprisonment, particularly in her communications regarding a plot to assassinate Queen Elizabeth I (known as the Babington Plot). In this cipher, letters of the alphabet were substituted with symbols, numbers, or other letters, following a specific pattern.

Vulnerabilities:

- **Frequency Analysis Still Effective:** Despite the complexity, substitution ciphers still maintain letter frequencies. Cryptanalysts like Thomas Phelippes could use frequency analysis to identify common letters (e.g., "e" or "t") based on their frequency in the ciphertext.
- **Contextual Clues:** Substitution ciphers are vulnerable if the context of the message can be guessed. With enough intercepted letters, cryptanalysts could recognize common phrases, deduce parts of the key, and gradually crack the entire cipher.
- **No Key Rotation:** Unlike more advanced ciphers (like the Vigenère cipher), Mary's method did not change keys during encryption, so patterns could accumulate across multiple messages, making decryption easier.

5.7 Charles Babbage and the Vigenère Cipher (1854):

Charles Babbage, the English mathematician known for conceptualizing the first mechanical computer, is credited with breaking the Vigenère cipher, though his work wasn't published. The decryption method relied on finding repeating patterns in ciphertext and reverse engineering the key.

Fixes:

- **Defense Against Frequency Analysis:** The Vigenère cipher improved upon simple substitution ciphers by using multiple shifting alphabets determined by a keyword. This made it much harder for cryptanalysts to use frequency analysis, as the same letter in the plaintext could be encrypted differently depending on its position.
- **Polyalphabetic Encryption:** The use of a keyword created a polyalphabetic cipher, where multiple alphabets are used to encrypt the message, making patterns more difficult to detect and adding complexity to the encryption.

Vulnerabilities:

- **Key Repetition:** If the keyword was too short or reused, cryptanalysts like Charles Babbage could detect repeating patterns in the ciphertext, allowing them to crack the cipher using methods like Kasiski examination and frequency analysis on individual parts.

- **Susceptibility to Cryptanalysis:** Despite its complexity, the Vigenère cipher remained vulnerable to cryptanalysis if the attacker could determine the length of the keyword. Once the length was known, the cipher could be reduced to several Caesar ciphers, which could be attacked individually.

5.8 World War I (20th Century)

The use of encryption expanded rapidly during World War I, with both sides employing ciphers to hide military communications. The Germans used the ADFGVX cipher, a polyalphabetic cipher that was broken by French cryptanalyst Georges Painvin, helping to turn the tide of the war.

5.9 World War II and Enigma (1930s–1940s):

Perhaps the most famous story of decryption comes from World War II. The Germans used the Enigma machine to encrypt their communications, which was thought to be unbreakable. However, Polish cryptanalysts first made headway in breaking Enigma in the 1930s, and later, British cryptographers at Bletchley Park, led by Alan Turing and others, developed techniques and machines to decrypt German messages. The breaking of Enigma was a major factor in the Allied victory.

5.10 Post-War and Digital Age (1940s–present)

Cold War and the Rise of Electronic Cryptography: During the Cold War, both sides engaged in extensive cryptographic efforts. The advent of computers and electronic communication introduced new challenges and opportunities in encryption and decryption. The National Security Agency (NSA) in the U.S. and its counterparts in other nations developed increasingly sophisticated cryptographic techniques.

5.11 Public-Key Cryptography (1970s):

A revolution in cryptography occurred with the invention of public-key cryptography by Whitfield Diffie and Martin Hellman in 1976. This method allowed secure communication over insecure channels without needing to share a secret key beforehand. The RSA algorithm, developed by Rivest, Shamir, and Adleman in 1977, became one of the most famous public-key cryptosystems.

5.12 Modern Cryptanalysis and Quantum Threats:

In recent years, cryptography and decryption have continued to evolve. The RSA algorithm and other methods like AES (Advanced Encryption Standard) are widely used today, but new threats are emerging, such as quantum computing. Quantum computers have the potential to break much of the encryption used today, prompting the development of quantum-resistant algorithms.

6 Cryptography

6.1 Encryption

Encryption is a crucial process in cryptography used to secure information by transforming plaintext into ciphertext, which can only be read or understood by those who have the decryption key. This process ensures the confidentiality and integrity of data during transmission or storage.

Modular Arithmetic Modular arithmetic is a system of arithmetic for integers where numbers wrap around upon reaching a certain value, known as the modulus. It operates similarly to the arithmetic on a clock, where after reaching the maximum number, the count starts again from zero. For example, in modular arithmetic with a modulus of 7, the number 8 is equivalent to 1, as $8 \bmod 7 = 1$. This system is fundamental in cryptography as it allows for efficient computation with large numbers and ensures that results remain within a manageable range. Modular arithmetic is used in various cryptographic algorithms to perform calculations securely and efficiently.

6.1.1 Modular Addition

Modular addition is a specific operation within modular arithmetic where two numbers are added together, and then the result is taken modulo a given number. Mathematically, if a and b are two integers, and m is the modulus, modular addition is expressed as: $(a + b) \bmod m$. This operation ensures that the sum stays within a defined range, making it suitable for various cryptographic applications.

For example, if we perform modular addition with $a = 5$, $b = 7$, and $m = 10$, the result is $((5 + 7) \bmod 10) \equiv (12 \bmod 10) = 2$. Modular addition is commonly used in encryption algorithms for key generation.

In other words, modular addition works like a clock arithmetic. Imagine a 12-hour clock—if it's 10 o'clock and you add 5 hours, it becomes 3 o'clock, not 15. You "wrap around" the clock. This wrapping around is what we call "modulo" in math. In encryption, we use modular arithmetic to keep numbers small and manageable, even when they get very big during calculations.

Encryption using Modular Addition: In modular arithmetic-based encryption, the decryption key is chosen such that it cancels out the encryption key under the given modulus. This guarantees that the original message can be retrieved correctly during decryption.

- The encryption operation can be described by the equation:

$$c \equiv (p + k) \pmod{26}$$

Where c represents the ciphertext (encrypted message), p is the plaintext (original message), and k is the encryption key. The modulo 26 ensures that the result stays within the range of letters in the alphabet.

- The decryption operation is given by:

$$p \equiv (c + \tilde{k}) \pmod{26}$$

Where $\tilde{k}$ is the decryption key, which is the additive inverse of k modulo 26. This ensures that the effect of the encryption key is undone when decrypting the message.

- The condition for correct decryption is:

$$(k + \tilde{k}) \pmod{26} = 0$$

This equation ensures that the encryption and decryption keys cancel each other out, allowing the original plaintext to be recovered.

The choice of 26 in modular arithmetic for encryption is based on the number of letters in the English alphabet. Since the alphabet consists of 26 letters (A to Z), when performing encryption and decryption, the numbers representing the letters need to wrap around after reaching 25 (where A = 0 and Z = 25).

In classical cryptography, such as the Caesar cipher or the Vigenère cipher, each letter of the alphabet is mapped to a number (e.g., A = 0, B = 1, ..., Z = 25). The operations are then performed modulo 26 to ensure that the results stay within the range of 0 to 25, which corresponds to the letters of the alphabet.

Thus, the modulus 26 is chosen because it matches the number of letters in the alphabet, ensuring that the encryption and decryption process remains consistent with the letter representation.

6.1.2 Modular Multiplication

Modular multiplication is a mathematical operation within modular arithmetic where two numbers are multiplied, and then the result is taken modulo a given number. Mathematically, if a and b are two integers, and m is the modulus, modular multiplication is expressed as $(a \times b) \pmod{m}$. This ensures

that the product remains within the defined range of integers from 0 to $m-1$, making it highly useful in cryptographic algorithms where operations need to be performed securely and efficiently over large numbers.

For example, if we perform modular multiplication with $a = 5$, $b = 7$, and $m = 10$, the result is $(5 \times 7) \bmod 10 = 35 \bmod 10 = 5$. This operation keeps the result within the range 0 to 9 by applying the modulus after multiplication.

Modular Multiplication in Cryptography: Modular multiplication is extensively used in cryptographic algorithms, especially in public-key cryptosystems such as RSA and Diffie-Hellman. These algorithms rely on modular exponentiation, which is built on modular multiplication, for securing communications.

- Consider the modular multiplication equation:

$$c \equiv (p \times k) \bmod m$$

Here, c is the ciphertext, p is the plaintext, and k is the key. The product is taken modulo m to ensure the result fits within a specific range, often determined by the size of the modulus.

- The decryption process generally involves finding the multiplicative inverse of k modulo m , denoted as k^{-1} , so that:

$$p \equiv (c \times k^{-1}) \bmod m$$

This equation ensures that the decryption key undoes the effect of the encryption key, thereby recovering the original message.

- The choice of the modulus m is critical in cryptography because it controls the size of the number space, typically chosen to be large prime numbers for enhanced security.

6.1.3 Modular Exponentiation

Modular exponentiation is a critical operation in modular arithmetic, particularly used in cryptographic algorithms such as RSA and Diffie-Hellman. It involves raising a number to a power, and then taking the result modulo a given number. Mathematically, if a is the base, e is the exponent, and m is the modulus, modular exponentiation is expressed as:

$$c \equiv a^e \bmod m$$

This operation ensures that the result of the exponentiation remains within a fixed range, making it ideal for handling large numbers securely and efficiently in cryptographic algorithms.

For example, if we perform modular exponentiation with $a = 5$, $e = 3$, and $m = 13$, the result is:

$$5^3 \mod 13 = 125 \mod 13 = 8$$

This operation keeps the result within the range 0 to $m-1$, allowing cryptographic algorithms to handle large exponents and bases without overflow or performance issues.

Modular Exponentiation in Cryptography: Modular exponentiation plays a fundamental role in many cryptographic systems. It provides security by creating operations that are computationally difficult to reverse without knowing certain secret values. For example, in public-key cryptography, such as RSA and Diffie-Hellman, modular exponentiation is used to generate and encrypt keys securely.

- In RSA encryption, the ciphertext c is calculated as:

$$c \equiv p^e \mod m$$

where p is the plaintext, e is the public key exponent, and m is the modulus, typically chosen as the product of two large primes. The result c is the encrypted message, which is securely transmitted.

- The decryption process uses the private key, which is another exponent d . The decryption is performed by computing:

$$p \equiv c^d \mod m$$

This reverses the encryption, recovering the original plaintext.

- The size of the modulus m ensures that the operations are performed within a finite field, providing a controlled number space for both encryption and decryption. The complexity of reversing modular exponentiation without the private key makes this operation ideal for securing communications.

6.1.4 Example to compare the three methods

Let's take one letter and encrypt it using three methods: **modular addition**, **modular multiplication**, and **modular exponentiation**. We'll use the same letter and mod (26) for simplicity. For this example, we'll use the letter "B" (which is 1 if $A = 0$) and different keys for each method.

Modular Addition We'll use the encryption formula:

$$c \equiv (p + k) \pmod{26}$$

Where:

- p is the plaintext letter ("B" = 1)
- k is the encryption key (let's choose $k = 5$)

Encryption:

$$c \equiv (1 + 5) \pmod{26} = 6$$

So, "B" is encrypted to the 6th letter, which is **G**.

Decryption: For decryption, we subtract the key:

$$p \equiv (c - k) \pmod{26} = (6 - 5) \pmod{26} = 1$$

So, we get back **B**.

Modular Multiplication The encryption formula is:

$$c \equiv (p \times k) \pmod{26}$$

Where:

- p is "B" = 1
- k is the encryption key (let's choose $k = 7$)

Encryption:

$$c \equiv (1 \times 7) \pmod{26} = 7$$

So, "B" is encrypted to the 7th letter, which is **H**.

Decryption: To decrypt using modular multiplication, we need the **modular inverse** of 7 under mod 26, which is 15 (since $7 \times 15 \pmod{26} = 1$).

The decryption formula is:

$$p \equiv (c \times \tilde{k}) \pmod{26}$$

$$p \equiv (7 \times 15) \pmod{26} = 1$$

So, we get back **B**.

Modular Exponentiation The encryption formula is:

$$c \equiv (p^k) \mod 26$$

Where:

- p is "B" = 1
- k is the encryption key (let's choose $k = 3$)

Encryption:

$$c \equiv (1^3) \mod 26 = 1$$

So, "B" remains **B** in this case.

Decryption: Since the encrypted value is still 1, there's no change after decryption. This method often works better with larger numbers but follows the same principle.

Summary of Results for the letter "B":

- **Modular Addition:** $B \rightarrow G$
- **Modular Multiplication:** $B \rightarrow H$
- **Modular Exponentiation:** $B \rightarrow B$

Each method has its own characteristics. **Modular addition** is like shifting letters, **modular multiplication** changes them more drastically, and **modular exponentiation** is often used for more complex encryption, especially with larger numbers.

6.1.5 Key VS modulo: a distinction to make

In cryptography, the key is the number that controls how encryption is applied, while the modulo is just a system boundary.

- **Modulo number:** This is the number used to limit the result (i.e., keep it within a specific range, like 0-25 for letters in the alphabet). In cryptographic algorithms, the modulo number is often predefined (e.g., mod 26 for alphabets, mod n for larger systems like RSA) and isn't considered a secret or a key
- **Key:** This is the secret or fixed number used in the encryption and decryption process. For example, in modular addition, it's the number added to the plaintext. In modular multiplication, it's the number multiplied by the plaintext. The key is crucial for both encryption and decryption.

6.2 How to Choose the Modulus

The choice of the modulus m in cryptographic systems is a critical aspect of ensuring security and efficiency. The selection of m typically depends on the type of cryptographic system being used.

6.2.1 Fixed Modulus in Classical Systems

In classical cryptographic systems, such as the Caesar cipher, the modulus m is selected based on the size of the symbol set. For example, when encrypting the English alphabet, m is often chosen to be 26, corresponding to the 26 letters in the alphabet. This ensures that after encryption, the ciphertext remains within the alphabetic range.

6.2.2 Larger Moduli in Modern Cryptography

In modern cryptographic systems, such as RSA and Diffie-Hellman, the modulus m is chosen to be a very large number, typically a large prime or the product of two large primes. This selection serves two purposes:

- **Security through Complexity:** The modulus m is often a large prime or a product of two large primes. This makes the problem of factorization or finding discrete logarithms computationally infeasible for modern computers. For instance, in RSA, m might be a 2048-bit number, making it practically impossible to factorize within a reasonable time frame.
- **Finite Field Arithmetic:** The large modulus ensures that all arithmetic operations occur within a finite field. This keeps the results within a manageable range and maintains mathematical consistency while preventing overflow during cryptographic operations.

Example in RSA Cryptography In RSA encryption, the modulus m is chosen as the product of two large prime numbers:

$$m = p \times q$$

where p and q are typically large 1024-bit primes. The product m , therefore, becomes a 2048-bit number. The larger the modulus, the more secure the encryption, as it becomes increasingly difficult to factorize m or break the encryption through brute-force methods. These numbers are so big that even modern computers can't easily figure out their factors (the numbers that multiply together to make m). Hence, making decryption practically impossible to do in a reasonable amount of time.

6.2.3 Balancing Security and Efficiency

The choice of m must balance security and computational efficiency. Larger values of m provide greater security but increase the computational complexity of encryption and decryption operations (not talking about attacks, talking about the operations themselves in a secure and normal environment).

Conversely, smaller values of m improve performance but reduce security. In practice, cryptographers select m large enough to meet security requirements without making encryption impractically slow.

Conclusion: In classical systems, m is often determined by the size of the symbol set. In modern cryptographic systems, m is chosen as a large prime or a product of primes to ensure security while maintaining computational efficiency. The correct choice of m is fundamental to the security of cryptographic algorithms.

6.3 Security through complexity

In cryptography, security through complexity means that we make the encryption process so complicated that it's very hard for anyone to break it. As we progress from modular addition to modular multiplication and then to modular exponentiation, the complexity of decryption increases significantly. In modular addition, decryption involves a simple reverse operation of subtracting a fixed key from the ciphertext, which is computationally trivial. Modular multiplication adds a layer of complexity, as decryption requires finding the modular inverse of the key, which is more challenging but still manageable with modern algorithms. However, modular exponentiation presents the highest complexity, as decrypting involves reversing an operation involving raising a number to a power and taking a modulus. This process requires advanced mathematical techniques and is computationally intensive, especially when dealing with large numbers, making it resistant to brute-force attacks and significantly increasing the overall security of the encryption system.

So, the complexity of the math makes the encryption secure!

6.4 Symmetric and asymmetric encryption

Symmetric and asymmetric encryption are two fundamental approaches to securing data, each with its distinct advantages and challenges.

Symmetric encryption uses a single key for both encryption and decryption, which makes it very efficient and fast, suitable for handling large volumes of

data quickly.

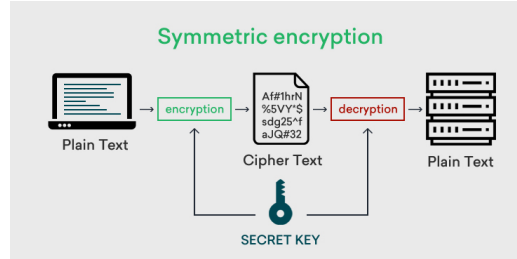


Figure 18: Symmetric encryption

Encryption key is called symmetric when k and $\tilde{K}$ are the same. However, the major drawback of symmetric encryption lies in the challenge of securely distributing the key between parties; if the key is intercepted or exposed, the security of the entire communication is compromised.

Asymmetric encryption employs a pair of keys: a public key for encryption and a private key for decryption. This method simplifies key distribution since the public key can be shared openly without compromising security. Only the holder of the private key can decrypt the data, enhancing security and facilitating digital signatures for authentication.

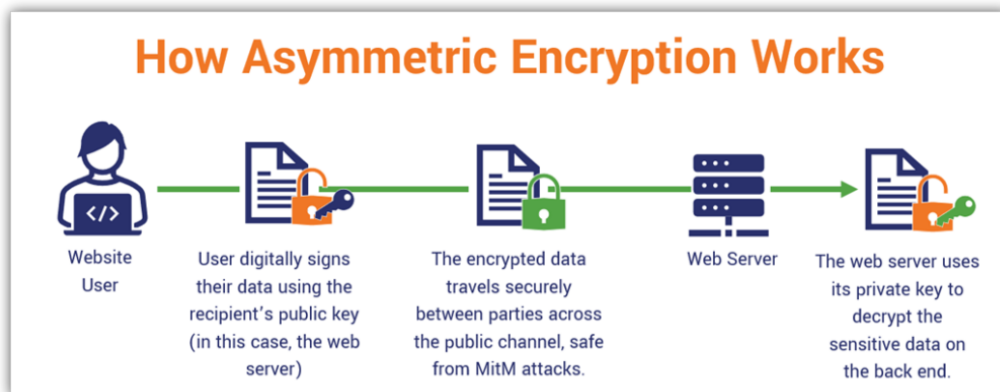


Figure 19: Asymmetric encryption

Encryption is said to be asymmetric when the sender knows the encryption key and the receiver knows a complementary decryption key. Encryption is called symmetric when the encryption key k and the decryption key $\tilde{K}$ are the same.

Discussion: Typically, in practice, a combination of both methods is used, leveraging symmetric encryption for data transfer speed and asymmetric encryption for secure key exchange.

Conclusion Public key encryption and asymmetric encryption are two terms that refer to the same concept. They are used interchangeably to describe a cryptographic method that employs a pair of mathematically related keys: a public key and a private key.

Here's a breakdown of the key points:

- **Public key:** This key is freely distributed and can be shared with anyone. It is used to encrypt data.
- **Private key:** This key is kept secret and is known only to the intended recipient. It is used to decrypt data that has been encrypted with the corresponding public key.

Key characteristics of public key encryption:

- **Asymmetric:** The keys used for encryption and decryption are different.
- **Non-reversible:** Data encrypted with a public key can only be decrypted with the corresponding private key, and vice versa.
- **Secure:** The security of the system relies on the secrecy of the private key. If the private key is compromised, the encryption becomes ineffective.

Common use cases for public key encryption:

- **Secure communication:** To establish secure connections between parties who have not previously shared a secret key.
- **Digital signatures:** To verify the authenticity of digital documents or messages.
- **Key exchange:** To securely exchange symmetric encryption keys over an insecure channel.

6.5 Symmetry and Asymmetry in modular encryption

In modular encryption, symmetry and asymmetry are reflected in how the keys are utilized for encryption and decryption.

In symmetric modular encryption such as those using modular addition, the same key is used for both encrypting and decrypting messages, which means the encryption method is reversible with the same key, illustrating the symmetric nature. While it might seem counter-intuitive given the previous discussion about the key and reverse key not always being identical, the core principle of symmetric encryption is that the same key is used for both encryption and decryption. In the case of modular addition, even though the exact values of the key and reverse key might differ, they are functionally equivalent. This is because the operation of adding or subtracting the key modulo the modulus effectively reverses the encryption process.

Therefore, despite the nuance in key usage, modular addition adheres to the fundamental characteristics of symmetric encryption: a single key is used for both encryption and decryption, making it a symmetric encryption method.

In asymmetric encryption: Conversely, modular multiplication and exponentiation are employed with a pair of keys: a public key for encryption and a private key for decryption. The keys are mathematically related but distinct, showcasing the asymmetric nature of the encryption system where different keys are used for each operation. Thus, while modular arithmetic underpins both symmetric and asymmetric methods, the key usage defines the encryption type.

6.6 Symmetric Encryption Algorithms

Symmetric encryption is a method where the same key is used for both encryption and decryption. It is efficient and fast, making it ideal for encrypting large amounts of data. The security of symmetric encryption relies on keeping the shared key secret between the communicating parties.

6.6.1 AES (Advanced Encryption Standard)

AES is one of the most widely used symmetric encryption algorithms today. It was standardized by the National Institute of Standards and Technology (NIST) in 2001 to replace the older DES algorithm. AES operates on fixed block sizes (128 bits) and supports key sizes of 128, 192, or 256 bits. Key features of AEs include:

- **High Security:** AES is highly resistant to all known forms of cryptanalysis, making it the standard for secure encryption.

- **Performance:** AES is efficient in both software and hardware, making it suitable for a variety of applications, from securing files to network encryption protocols.
- **Widely Adopted:** AES is used in protocols such as TLS (for secure web browsing), IPsec (for secure network communications), and disk encryption systems like BitLocker.

6.6.2 DES (Data Encryption Standard) and 3DES

DES is of historical significance, but now considered insecure due to vulnerabilities. DES was the previous encryption standard before AES and operates on 64-bit blocks using a 56-bit key. It was widely used in the 1970s and 1980s but is now considered insecure due to its small key size, which makes it vulnerable to brute-force attacks.

6.6.3 3DES (Triple DES)

To address the security weaknesses of DES, 3DES was developed. It applies the DES algorithm three times in a row with either two or three different keys, increasing security without requiring a new algorithm. While 3DES provides better security than DES, it is significantly slower than AES and is being phased out in favor of more modern algorithms.

6.6.4 Applications of Symmetric Encryption

Symmetric encryption is used in various scenarios, including:

- **Data Encryption:** For encrypting sensitive data stored on disk or transmitted over a network.
- **Secure Communications:** Many secure communication protocols, such as VPNs (using IPsec) and HTTPS (using TLS), rely on symmetric encryption for protecting data.
- **File Encryption:** Tools like VeraCrypt and WinRAR use symmetric encryption to protect files and folders.

Overall, symmetric encryption plays a crucial role in modern cryptographic systems due to its balance of security and efficiency. Understanding algorithms like AES and the history of DES helps build a solid foundation in cryptography.

6.7 Asymmetric encryption Algorithms

Asymmetric encryption, also known as public-key cryptography, uses two different keys: a public key for encryption and a private key for decryption. Unlike symmetric encryption, where both parties share the same key, asymmetric encryption allows secure communication without sharing the private key. This makes it ideal for secure key exchanges and digital signatures, where confidentiality and authenticity are critical.

6.7.1 RSA (Rivest-Shamir-Adleman)

RSA is one of the most widely used asymmetric encryption algorithms. It was introduced in 1977 and remains a cornerstone of public-key cryptography. RSA uses large prime numbers to generate a pair of keys: one public and one private. The public key is used to encrypt data, while the private key is used to decrypt it.

Key features of RSA include:

- **Encryption and Digital Signatures:** RSA is used for both encrypting data and creating digital signatures, where the private key signs a message and the public key verifies the signature.
- **Security:** RSA's security is based on the difficulty of factoring large numbers, a problem that becomes harder as key sizes increase. Common key lengths are 2048 and 4096 bits.
- **Applications** RSA is widely used in secure protocols like TLS (for HTTPS), email encryption (PGP), and digital certificates.

6.7.2 ECC (Elliptic Curve Cryptography)

Elliptic Curve Cryptography is a modern approach to public-key encryption that provides the same level of security as RSA with much smaller key sizes. ECC is based on the mathematics of elliptic curves over finite fields, making it highly efficient and secure.

Key features of ECC include:

- **Efficiency:** ECC requires smaller key sizes compared to RSA to achieve the same level of security. For example, a 256-bit ECC key provides comparable security to a 3072-bit RSA key. This makes ECC faster and more resource-efficient, particularly in environments with limited processing power.

- **Security:** ECC is considered highly secure and resistant to known attacks, especially for modern applications requiring high performance and security.
- **Applications:** ECC is used in secure communication protocols (such as TLS), digital signatures (ECDSA), and mobile device encryption due to its low computational overhead.

RSA vs ECC Common algorithms used in public key encryption include RSA, which relies on the mathematical properties of large prime numbers, and ECC (Elliptic Curve Cryptography), which offers strong security with smaller key sizes.

- **RSA:** While RSA remains widely used, its large key sizes make it less efficient in terms of processing power and speed. It is still highly secure but is slowly being replaced by more efficient algorithms.
- **ECC:** ECC is gaining popularity due to its efficiency and smaller key sizes, which are particularly useful in mobile devices, IoT, and other resource-constrained environments.

6.7.3 Applications of Asymmetric Encryption

Asymmetric encryption is primarily used in scenarios that require secure communication, key management, and authentication. Some key applications include:

- **Key Exchange:** Asymmetric encryption allows two parties to securely exchange keys over an insecure channel. For example, in the TLS protocol, RSA or ECC is used to exchange symmetric keys, which are then used for faster encryption.
- **Digital Signatures:** Digital signatures use the private key to sign data, ensuring that the data has not been tampered with. The recipient can verify the signature using the public key, confirming both the integrity and authenticity of the message.
- **Public Key Infrastructure (PKI):** RSA and ECC are central to PKI systems, which issue digital certificates to verify the identity of parties in secure communications (such as HTTPS websites).

6.8 Hashing functions

Hashing functions are mathematical algorithms that take an input (of any size) and produce a fixed-size output, known as a hash or digest. This output is typically much smaller than the input, but it's designed to be unique for that specific input.

6.8.1 How Hashing Works?

Input: A piece of data (e.g., a file, a password, a message) is fed into the hashing function. Processing: The function performs a series of complex mathematical operations on the input. Output: The result is a fixed-length hash value. Key Properties of Hash Functions Deterministic: The same input will always produce the same hash. Efficient: Hashing functions should be computationally efficient to calculate. Pre-image Resistant: It should be difficult to find the original input (pre-image) given the hash value. Collision Resistant: It should be difficult to find two different inputs that produce the same hash value.

6.8.2 Role of Hashing in Cybersecurity

Hashing functions play a crucial role in cybersecurity for several reasons:

- Password Storage: Hashing is used to store passwords securely. Instead of storing the plain-text password, the hashed version is stored. If a database is compromised, attackers cannot easily access the original passwords.
- Data Integrity: Hashing can be used to verify the integrity of data. A hash is calculated for a file or message. If the file is modified, the hash will change, indicating corruption or tampering.
- Digital Signatures: Hashing is used in digital signatures to verify the authenticity of a document or message. The sender calculates the hash of the document, signs it with their private key, and appends the signature to the document. The recipient can verify the signature by calculating the hash of the document and comparing it to the one signed by the sender.
- Blockchain Technology: Hashing is fundamental to blockchain technology. Each block in a blockchain contains a hash of the previous block, creating a chain of blocks that is difficult to tamper with.

While hashing functions are essential tools in cybersecurity, they are not cryptographic algorithms designed to keep secrets. Here's why:

- One-way function: Hashing functions are one-way, meaning it's virtually impossible to reverse the process and recover the original input (pre-image) from the hash value. This makes them unsuitable for encryption, where the ability to decrypt the ciphertext is crucial.
- Collision resistance: While collision resistance is important for cryptographic hash functions, it's not the primary goal. In password storage, for example, the main concern is preventing unauthorized access to the original password.

In summary, hashing functions are essential tools in cybersecurity, providing security for password storage, data integrity, digital signatures, and blockchain technology. Their ability to create unique, fixed-size outputs while maintaining pre-image and collision resistance makes them invaluable for protecting sensitive information. *However, they are not designed to encrypt data and keep it secret.*

6.8.3 MD5 (Message-Digest 5) Hashing Function

Message digests, or hash functions, are one-way operations that take an input of any size and produce a fixed-length output, known as a digest. One of the most well-known examples is MD5 (Message Digest 5), a one-way cryptographic function, which generates a fixed-length digest from any input and is commonly used for message authentication.

Example: If the input message is *"Hello, world!"*, the MD5 hash would be: *bd7619358775f7c0890b10310d167090*

MD5: The algorithm takes as input a message of arbitrary length and produces as output a 128-bit 'fingerprint' or 'message digest' of the input. It is conjectured that it is computationally infeasible to produce two messages having the same message digest, or to produce any message having a given prespecified target message digest. The MD5 algorithm is intended for digital signature applications, where a large file must be 'compressed' in a secure manner before being encrypted with a private (secret) key under a public-key cryptosystem such as RSA.

Is MD5 a cryptographically secure method?

The goal of any message-digest function is to produce digests that appear

to be random. To be considered cryptographically secure, the hash function should meet two requirements:

- It is impossible for an attacker to generate a message matching a specific hash value.
- It is impossible for an attacker to create two messages that produce the same hash value.

MD5 can still be used for integrity protection in certain cases, such as error detection and verification within protocols. However, it is no longer considered secure for cryptographic purposes. MD5 is vulnerable to collision attacks, where two different inputs can produce the same hash. As a result, it is no longer used for cryptographic authentication. Although MD5 was once widely adopted, modern algorithms like SHA-256 or SHA-3 are now recommended for secure applications.

6.8.4 SHA (Secure Hash Algorithm)

The Secure Hash Algorithm (SHA) is a family of cryptographic hash functions designed to provide data integrity and security. Hash functions take an input (or "message") and produce a fixed-length string of characters, which appears random. Even a small change in the input will produce a significantly different output. SHA functions are widely used in various cryptographic applications such as digital signatures, certificates, and data integrity verification.

Applications of SHA SHA algorithms are primarily used for:

- **Data Integrity Verification:** Ensures that data has not been altered or corrupted by generating a hash before and after transmission.
- **Digital Signatures:** The hash of a message is signed, allowing the recipient to verify both the signature and the integrity of the message.
- **Password Hashing:** Securely stores passwords by hashing them, so that even if the database is compromised, passwords are not easily recoverable.
- **Blockchain Technology:** Cryptocurrencies like Bitcoin use SHA-256 for mining and securing transactions.

Finally hashing functions are a family of algorithms in cryptography that ensures data integrity, protects digital signatures and secures communications in modern systems.

6.9 Key Exchange Protocols for Symmetric Encryption

Key exchange protocols are cryptographic methods that allow two parties to securely share a secret key over an insecure channel, such as the internet. This shared key can then be used for symmetric encryption, which is faster and more efficient than public-key encryption. The key exchange ensures that no third party can intercept or deduce the key, even if they are monitoring the communication.

The most fundamental key exchange protocol is the Diffie-Hellman protocol, and its modern counterpart, Elliptic Curve Diffie-Hellman (ECDH), offers enhanced efficiency and security for systems requiring smaller key sizes and high performance.

6.9.1 Diffie-Hellman (DH)

The Diffie-Hellman (DH) key exchange protocol, introduced in 1976, was one of the first practical methods for secure key exchange. It allows two parties, who may not have prior knowledge of each other, to generate a shared secret over a public channel.

How Diffie-Hellman Works:

- The two parties (commonly referred to as Alice and Bob) agree on a large prime number p and a base (or generator) g , both of which are public.
- Each party selects a private key. Alice selects a secret value a , and Bob selects a secret value b . These private values are never shared.
- Alice computes $A = g^a \bmod p$ and sends A to Bob. Bob computes $B = g^b \bmod p$ and sends B to Alice.
- Both Alice and Bob can now compute the shared secret:
 - Alice computes $s = B^a \bmod p$
 - Bob computes $s = A^b \bmod p$
- Since both calculations result in the same value, s , this becomes the shared secret key.

Example: The Diffie-Hellman key exchange allows two parties, Alice and Bob, to securely establish a shared secret over a public channel. Below is a step-by-step example of how the process works.

1. Public Information:

- Prime number, $p = 23$.
- Base (or generator), $g = 5$.

Both p and g are publicly known.

2. Private Keys (Secret Values):

- Alice chooses a private key $a = 6$.
- Bob chooses a private key $b = 15$.

3. Exchange Public Values:

Each party computes their public value and sends it to the other:

- Alice computes her public value:

$$A = g^a \mod p = 5^6 \mod 23 = 15625 \mod 23 = 8$$

- Bob computes his public value:

$$B = g^b \mod p = 5^{15} \mod 23 = 30517578125 \mod 23 = 19$$

Alice sends $A = 8$ to Bob, and Bob sends $B = 19$ to Alice.

4. Calculate the Shared Secret:

Now, each party computes the shared secret using the received public value and their own private key:

- Alice computes the shared secret:

$$S = B^a \mod p = (g^b)^a \mod 23 = 19^6 \mod 23 = 47045881 \mod 23 = 2$$

- Bob computes the shared secret:

$$S = A^b \mod p = 8^{15} \mod 23 = 35184372088832 \mod 23 = 2$$

Both Alice and Bob now share the same secret key, $S = 2$.

Even though a third party can observe the public values p , g , A , and B , they cannot easily compute the shared secret without knowing the private keys a or b .

6.9.2 Elliptic Curve Diffie-Hellman (ECDH)

Elliptic Curve Diffie-Hellman (ECDH) is a modern variant of the Diffie-Hellman protocol that leverages the mathematics of elliptic curves to provide the same level of security with much smaller key sizes. This makes ECDH faster and more efficient, particularly in environments where processing power, memory, and bandwidth are limited, such as mobile devices, IoT devices, and cloud computing.

How ECDH Works:

- Both parties agree on a set of elliptic curve parameters, including a base point G on the elliptic curve.
- Alice and Bob each generate a private key. Alice's private key is a , and Bob's private key is b . These values are kept secret.
- Each party generates a public key by multiplying their private key with the base point G on the elliptic curve:
 - Alice computes her public key $A = aG$ and sends it to Bob.
 - Bob computes his public key $B = bG$ and sends it to Alice.
- Both parties can now compute the shared secret by multiplying the other party's public key with their own private key:
 - Alice computes $s = aB = a(bG)$.
 - Bob computes $s = bA = b(aG)$.
- Since both calculations result in the same value, s , this becomes the shared secret key.

Key Features of ECDH:

- **Efficiency:** ECDH provides strong security with much smaller key sizes than traditional Diffie-Hellman. For example, a 256-bit key in ECDH offers the same level of security as a 3072-bit key in standard DH, making it ideal for high-performance, low-power environments.
- **Security:** The security of ECDH is based on the difficulty of solving the Elliptic Curve Discrete Logarithm Problem (ECDLP), which is significantly harder than the standard discrete logarithm problem, offering more robust protection against attacks.

- **Applications:** ECDH is widely used in modern cryptographic protocols like TLS 1.3, SSH, Bitcoin, and secure messaging apps.

Advantages of ECDH over DH:

- **Smaller Key Sizes:** ECDH requires smaller key sizes to provide the same level of security, reducing computational and storage overhead.
- **Faster:** The elliptic curve operations used in ECDH are more efficient, making the protocol faster than traditional Diffie-Hellman, especially on devices with limited processing power.
- **Stronger Security:** ECDH provides enhanced security due to the hardness of the elliptic curve discrete logarithm problem.

6.10 Key Exchange Protocols for Asymmetric Encryption

Who Decides on the Key Values? In asymmetric encryption, each client generates their own public-private key pair. The public key is freely distributed, while the private key remains secret.

How the Other Client Receives the Key Value

- **Public Key Infrastructure (PKI):** Typically, a PKI is used to manage and distribute public keys. This involves a trusted authority (Certificate Authority or CA) that verifies the identity of clients and issues digital certificates containing their public keys.
- **Direct Exchange:** In some cases, clients can exchange public keys directly, but this is less secure as there's no guarantee of the authenticity of the public key.

Example using a PKI

- **Obtain Certificates:** Both clients obtain digital certificates from a CA, proving their identity.
- **Exchange Certificates:** The clients exchange their certificates, which contain their public keys.
- **Verify Certificates:** Each client verifies the authenticity of the other's certificate using the CA's public key.

- **Encrypt and Decrypt:** To send a message, one client encrypts it using the recipient's public key. The recipient decrypts it using their private key.

Key Points:

- **No Key Exchange:** Unlike symmetric encryption, there's no need for a secure channel to exchange a shared key.
- **Public Key Distribution:** Public keys can be freely distributed, making it easier to establish secure communication with unknown parties.
- **Digital Signatures:** Asymmetric encryption can also be used to create digital signatures, which verify the authenticity and integrity of messages.

6.10.1 Exchanging modulus with public keys?

In asymmetric encryption, clients typically do not share their modulus. The modulus is a crucial component of the cryptographic algorithm used in asymmetric encryption (e.g., RSA). It is kept secret along with the private key. Sharing the modulus would weaken the security of the system. In fact, the public key, which is shared, includes the modulus and an exponent. However, the modulus is also part of the private key, which remains secret.

Security: The security of the system relies on the difficulty of factoring the modulus. If the modulus were public, it would be easier for attackers to factor it, potentially compromising the security of the system.

Key Generation: The modulus is typically generated as a product of two large prime numbers. Keeping the prime factors secret is essential for maintaining the security of the encryption.

Therefore, while the public key includes the modulus, it's important to remember that the modulus is also a component of the private key, and sharing it would compromise the security of the asymmetric encryption system.

7 Security for IoT

Internet of Things, or IoT, is the paradigm of building a network of connected objects/devices around us to provide seamless communication and contextual services offered by them. The concept is to create communication between the devices and the cloud and the devices themselves. The Internet of Things, or IoT, is a system of interrelated computing devices, that are provided with unique identifiers (UIDs) and have the ability to transfer data over a network and interact with each other without requiring human-to-human or human-to-machine-interaction.



Figure 20: The Internet of Things

The different types of "Things": There are many different types of IoT devices/objects, or "*Things*", ranging from simple sensors to more advanced devices. These devices, ranging from everyday objects like smartphones and thermostats to specialized equipment such as medical implants and agricultural sensors, are equipped with unique identifiers (IP addresses) and can communicate over networks.

Examples of IoT devices include:

- Wearables: Smartwatches, fitness trackers
- Medical devices: Insulin pumps, heart monitors
- Home automation: Smart thermostats, security systems
- Industrial equipment: Assembly line sensors, factory robots
- Transportation: Connected cars, autonomous vehicles

IoT Applications: IoT can be used in almost all domain of applications like smart homes, smart cities, smart farming, smart retail, smart grids, industrial internet, connected cars, transportation, smart supply chain, connected health. . .

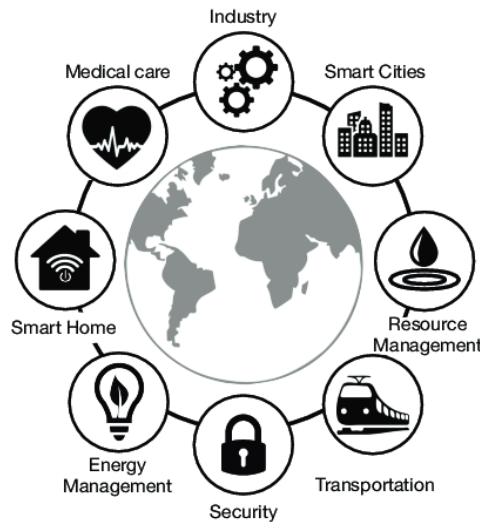


Figure 21: IoT applications

7.1 Security challenges for IoT

Security for the Internet of Things (IoT) presents unique challenges due to the nature of these devices and the environments in which they operate.

- **Resource Constraints:** Many IoT devices are small, inexpensive, and designed for specific, often minimal, tasks. This leads to limited computational power, memory, and battery life. Implementing traditional security measures like encryption or advanced authentication systems becomes difficult, as these devices might lack the necessary resources to handle them efficiently.
- **Scalability:** The sheer number of IoT devices (expected to reach billions) introduces scalability issues. Managing security protocols, authentication, and data encryption across large-scale networks becomes extremely complex. Each device needs to be authenticated and tracked securely, requiring sophisticated and adaptable security management systems.

- **Diverse and Heterogeneous Ecosystem:** IoT devices come from different manufacturers, with varying standards, architectures, and protocols. This lack of standardization makes it difficult to ensure that all devices across the network are uniformly secure. Vulnerabilities in one type of device may expose the entire network to attacks.
- **Limited Update Mechanisms** Many IoT devices lack easy mechanisms for software or firmware updates, leaving them vulnerable to security flaws discovered after they have been deployed. Even when updates are possible, users may not apply them, or manufacturers might stop providing updates, creating a long-term security risk.
- **Weak Authentication and Authorization:** Many IoT devices lack strong authentication and authorization mechanisms. Default or weak passwords, lack of multi-factor authentication, or minimal user control over access permissions make it easy for attackers to gain unauthorized access to devices.
- **Data Privacy:** IoT devices often collect vast amounts of sensitive data, including personal, health, or location information. If this data is transmitted or stored without proper encryption or privacy controls, it becomes vulnerable to interception or misuse by malicious actors.
- **Communication Security:** IoT devices often rely on wireless communication (Wi-Fi, Bluetooth, Zigbee, etc.), which can be vulnerable to interception, eavesdropping, or man-in-the-middle attacks. Ensuring secure communication channels is vital, but some protocols lack robust security features, or they're implemented incorrectly.
- **Insecure APIs and Cloud Services:** IoT devices often rely on cloud services and APIs for management, data processing, and storage. If these interfaces are insecure, attackers could exploit them to gain unauthorized access to devices or user data.
- **Botnets and DDoS Attacks** Compromised IoT devices can be used in botnets, as seen in the Mirai botnet attack ⁶, which weaponized thousands of insecure IoT devices to launch distributed denial-of-service (DDoS) attacks. Since many devices have weak defenses, they are easily co-opted into such networks, amplifying the threat.
- **Supply Chain Vulnerabilities:** IoT devices often rely on components from various manufacturers. Security vulnerabilities could be introduced

⁶[https://en.wikipedia.org/wiki/Mirai_\(malware\)](https://en.wikipedia.org/wiki/Mirai_(malware))

anywhere along the supply chain, either intentionally (e.g., through hardware backdoors) or accidentally (e.g., through poorly tested software).

- **Long Lifecycle and Legacy Devices** IoT devices often remain in use for many years, and older devices might lack the ability to keep up with evolving security standards. Maintaining security for legacy IoT devices, especially when they're critical infrastructure, is a major challenge.

Addressing these challenges requires a comprehensive approach involving strong encryption, robust authentication, secure development practices, regular software updates, and user education. Additionally, governments and organizations need to work together to create security standards, enforce regulations, and promote awareness of the importance of IoT security.

7.2 IoT Security Vulnerabilities Layers

The Internet of Things (IoT) plays critical roles across various sectors, including the industrial domain, connected homes, and consumer wearables. While IoT's expansion brings efficiency and new capabilities, it also introduces a wide array of privacy and security challenges. Some vulnerabilities are ubiquitous across all domains, while others are unique to specific use cases. Understanding these vulnerabilities is essential to mitigate security risks and ensure a robust IoT ecosystem.

7.2.1 Hardware vulnerabilities

IoT devices, being deeply embedded systems, are vulnerable to several hardware-specific attacks that exploit physical properties of the device itself.

- **Side-channel attacks:** These attacks exploit information leaked through physical channels (e.g., power consumption, electromagnetic radiation) rather than direct software or data manipulation. Attackers may measure variations in power or electromagnetic emissions to infer sensitive data, such as cryptographic keys.
- **Hardware Trojans:** Malicious modifications to the hardware during manufacturing or integration can introduce hidden backdoors. These Trojans can allow unauthorized access to the device or its data, sabotaging operations or creating vulnerabilities that are difficult to detect at the software level.

- **Physical tampering:** Since IoT devices are often deployed in uncontrolled environments, they can be physically accessed and tampered with. Attackers could reverse-engineer the hardware or modify components to interfere with functionality.

7.2.2 Software vulnerabilities

Software vulnerabilities in IoT devices are among the most exploited due to common programming and configuration errors. These vulnerabilities can be introduced during development, deployment, or in the firmware of IoT devices.

- **Buffer overflows:** IoT devices frequently operate with constrained memory. A buffer overflow occurs when more data is written to a memory buffer than it can hold, which can overwrite adjacent memory and potentially execute arbitrary code. Attackers exploit these overflows to hijack control over the device.
- **Injection attacks:** Poorly validated user input, whether in web applications or device interfaces, can lead to injection vulnerabilities (e.g., SQL injection, command injection). Attackers may send malicious payloads that are executed as part of legitimate commands, allowing unauthorized control or access.
- **Insecure APIs:** IoT devices often interact with cloud services and other systems via APIs. If these APIs lack proper authentication or encryption mechanisms, attackers can intercept or manipulate data exchanges, leading to data breaches or device manipulation.
- **Unpatched vulnerabilities:** IoT devices frequently use outdated software or firmware, especially when updates are not automated or manufacturer support is limited. These unpatched vulnerabilities can provide attackers with an entry point to compromise the system.

7.2.3 Network vulnerabilities

Since IoT devices rely heavily on network communication, many vulnerabilities stem from weaknesses in network protocols and configurations.

- **Weak authentication:** Devices often use default or weak credentials, making them susceptible to brute-force or dictionary attacks. Attackers can gain unauthorized access to the network and manipulate devices.

- **Insecure communication protocols:** IoT devices frequently use lightweight or outdated protocols (e.g., HTTP, MQTT) that lack robust encryption mechanisms. Without encryption, data transmitted between devices and cloud services can be intercepted, allowing attackers to eavesdrop or alter communications.
- **Denial of Service (DoS) attacks:** IoT devices are vulnerable to DoS or Distributed Denial of Service (DDoS) attacks due to limited processing power. Attackers can overwhelm devices or entire networks, disrupting services or causing devices to malfunction.
- **Man-in-the-Middle (MitM) attacks:** Weak encryption or lack of proper network segmentation can allow attackers to intercept and alter communications between IoT devices. By positioning themselves between the device and the network, attackers can manipulate data or even impersonate the device.

7.3 Security Considerations for IoT

Since IoT devices are vulnerable to a range of cyber threats (e.g., man-in-the-middle attacks, eavesdropping, data breaches), security in IoT protocols needs careful consideration:

- **End-to-End Encryption:** To protect sensitive data, especially in health and industrial IoT, protocols should offer strong encryption like AES.
- **Authentication Mechanisms:** Robust authentication (e.g., mutual authentication, multi-factor) is necessary to prevent unauthorized access.
- **Lightweight Security Solutions:** Protocols like CoAP and MQTT focus on reducing overhead while maintaining some level of encryption and authentication.
- **Firmware Updates:** Protocols should support secure and easy-to-apply firmware updates to prevent long-term vulnerabilities.

For IoT ecosystems, choosing the right protocol depends on factors like device capabilities, power consumption needs, communication range, and the specific use case, balancing these with security requirements.

On October 21, 2016, a distributed denial-of-service (DDoS) attack targeted Dyn, a provider of Domain Name System (DNS) management services. The attackers aimed to disrupt Dyn's operations using previously available malicious

code. The Mirai botnet, which utilized compromised IoT devices, generated overwhelming traffic, rendering Dyn and many of its clients offline. The attack occurred in two phases, first affecting the U.S. East Coast before escalating to a global scale. As Dyn's DNS servers struggled to refresh their content during the attack, the impact intensified.

A February 2017 article on Security Ledger highlighted that this incident resulted in at least 14,000 internet domains ceasing to use Dyn's services, causing significant financial losses for both the affected domains and Dyn itself.

To understand this incident, it's essential to define a distributed denial-of-service attack. According to Wikipedia, such attacks aim to make a machine or network resource unavailable to its intended users, often through the temporary interruption of services. Although these attacks have existed for a long time, their sophistication has increased over the years. Attackers typically attempt to saturate a network or host, preventing legitimate users from accessing services. Amplification attacks, which exploit certain protocols to generate substantial traffic with minimal input, exemplify this strategy.

The IoT landscape poses unique challenges for DDoS attacks. Mirai showcased how easily attackers could exploit IoT devices to form a large botnet, highlighting the vulnerability of these devices due to poor security practices. Many users do not apply software updates or configure security settings properly, creating opportunities for malicious actors to compromise devices.

However, the responsibility does not lie solely with consumers. Device manufacturers also share the blame for failing to implement adequate security measures. For instance, some vendors have created devices with hardcoded passwords, leaving them susceptible to widespread exploitation. Despite lessons learned from past incidents, many still employ insecure practices, allowing attackers to replicate their methods across various devices.

Once a botnet is established, the attacker must manage and coordinate the compromised devices, often referred to as "zombies." Botnets can vary in size, potentially comprising thousands of compromised hosts. Command and control channels are used to issue commands and gather information from these devices. For example, the Mirai botnet utilized a hardcoded listener, which can help defenders identify suspicious activity on their networks.

The implications of such powerful botnets extend beyond mere disruptions; they threaten the very infrastructure that supports modern connectivity, including smart cities and dynamic energy systems. Designers of these systems must prioritize security to prevent easy takeovers by malicious actors while also building resilience into their infrastructure to withstand potential attacks.

As the reliance on interconnected systems grows, it becomes critical for

manufacturers to adopt best practices for securing their devices. Similarly, system designers need to implement defensive strategies that safeguard against future DDoS attacks, ensuring the integrity of the services that society depends on. This incident underscores the importance of robust security measures and proactive risk management in the evolving landscape of IoT and connectivity.

Connected vehicles communicate with each other using a communication standard called Dedicated Short Range Communications (DSRC). This system allows vehicles to share important information, such as when a car is stopping or if there are hazards on the road ahead, improving safety. You may have also heard of autonomous or self-driving vehicles. These vehicles rely on their own sensors, like LIDAR (Light Detection and Ranging), to understand their surroundings.

Both communication and self-driving technologies are advancing at the same time, which raises security concerns. Adding new communication and sensing capabilities increases the risk of vulnerabilities, especially since modern cars are already complex, with some having over 100 million lines of code. The complexity of these systems often leads to mistakes that can be exploited by attackers. Moreover, connected vehicles interact with various systems. Three main concepts are crucial to understanding these connections:

- **Vehicle-to-Vehicle (V2V)** communication, where vehicles exchange safety messages.
- **Vehicle-to-Infrastructure (V2I)**, where the infrastructure, such as traffic lights, informs vehicles of upcoming dangers.
- **Vehicle-to-Everything (V2X)**, where vehicles detect pedestrians or other objects to help prevent accidents.

These technologies rely on DSRC (Dedicated Short Range Communications), defined by IEEE standards, but connected vehicles also use Bluetooth, NFC, Wi-Fi, and even cellular networks, making them a prime target for cyberattacks.

Connected vehicles face multiple security threats from a variety of actors. A typical "black hat" hacker may attempt to exploit vulnerabilities in the system simply to cause disruption or test the damage they can inflict. In some cases, minor actions such as lowering a vehicle's windows can be achieved remotely. However, more serious research has shown that attackers could manipulate primary vehicle functions, leading to dangerous outcomes.

One key concern is the potential for ransomware attacks, where hackers could deny the vehicle owner access to their car until a ransom is paid. Such attacks could extend beyond individual vehicles. In extreme cases, a nation-state could launch coordinated cyberattacks, paralyzing traffic across major cities or regions.

Security threats in connected vehicles

As self-driving and connected vehicles evolve in the era of the Internet of Things (IoT), new integration points are being developed, increasing the attack surface. For example, insurance companies are using trackers that connect to vehicles' onboard diagnostics (OBD-II) ports, while manufacturers are collaborating with smart home technologies like Amazon's Alexa for voice control integration. These integration points, while convenient, create additional potential vulnerabilities. If a connected IoT system is compromised, hackers may gain access to vehicles through this backdoor.

Cloud services are also becoming a critical part of the connected vehicle ecosystem. Vehicles and traffic management systems will increasingly rely on cloud-based systems, providing further opportunities for attackers to exploit. This complex ecosystem includes traffic management centers that communicate with roadside units (RSUs) and vehicles through technologies such as 3G, 4G, and satellite communication, adding layers of communication channels that could be targeted.

To address these challenges, security measures have to be defined to ensure the integrity and authenticity of data within the connected vehicle ecosystem. One such initiative is the Security Credential Management System (SCMS), which uses a public key infrastructure (PKI) to issue credentials to vehicles. These credentials are designed to protect data privacy and ensure that operators cannot trace certificates back to specific vehicles.

As connected and autonomous vehicle technologies continue to develop, security will remain a critical concern. New safeguards are being implemented, but the evolving complexity of these systems demands ongoing attention to protect against both current and future cyber threats.

Microgrids are becoming an increasingly popular energy solution for both residential and industrial consumers as they adopt clean energy technologies. These systems integrate various Distributed Energy Resources (DERs) such as solar panels, wind turbines, and batteries. By linking these resources, microgrids provide a localized and efficient energy solution. While microgrids are typically connected to the main utility grid, they also have the capability to operate independently, or in "island mode," when necessary. Microgrids function through Supervisory Control and Data Acquisition (SCADA) systems that manage and optimize energy generation and distribution. A typical microgrid features multiple energy sources and serves both residential and industrial customers, making it a prime example of a Cyber-Physical System (CPS). CPS refers to systems that merge the physical and digital worlds, and microgrids are a key subset of the Internet of Things (IoT) due to their reliance on advanced analytics, predictive modeling, and embedded controllers.

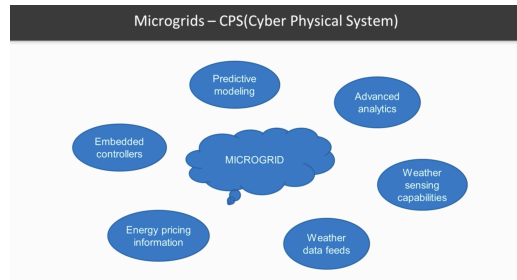


Figure 22: Microgrids - Cyber Physical System (CPS)

For instance, a microgrid often receives data from the local utility regarding energy prices, allowing it to make autonomous decisions about when to generate its own power or purchase it from the grid. Additionally, microgrids may integrate weather data to predict how much energy will be generated by solar panels or wind turbines at different times. This fusion of real-time data, predictive modeling, and physical energy generation illustrates the complexity of CPS, which requires expertise across multiple engineering disciplines, such as control theory, thermodynamics, and digital design. Cyber-Physical Systems (CPS) are particularly intriguing because they combine sensors, actuators, and control systems, which may or may not be connected to the Internet. However, this blending of the cyber and physical domains increases their vulnerability to cyberattacks. Developers of CPS, including microgrid developers, must therefore focus on integrating cybersecurity measures with traditional control system engineering to safeguard against potential attacks. One critical concern is whether a cyberattack could disable safety systems designed to protect the physical infrastructure.

For example, in December 2014, a cyberattack on a German steel mill's industrial control system resulted in significant physical damage. Such incidents highlight the need for microgrid developers to adopt a "defense-in-depth" approach. This means building systems that integrate both cybersecurity and safety controls from the outset, ensuring they are resilient against faults and capable of preventing or minimizing damage in the event of a cyberattack. Developers must also consider how attackers might exploit engineering principles to their advantage and evolve safety controls to include cyber defenses alongside traditional measures like fault tolerance and redundancy. Physical failsafes, along with active monitoring and mitigation, should be implemented to detect and disrupt malicious activities that could harm the system. In summary, microgrids represent a critical component of Cyber-Physical Systems, combining energy generation with advanced digital controls. As these systems continue to grow in importance, developers must address the

challenges posed by cyber threats to ensure the safety and reliability of this new energy infrastructure. Next, we will explore another form of CPS—the use of municipal drones.

Drones, though often seen as consumer toys, offer significant capabilities for smart city applications, such as firefighting, police operations, and emergency response. As cities and citizens begin to rely on drones, ensuring their security and availability becomes a critical concern. However, several challenges exist in securing these systems.

First, many consumer drone manufacturers prioritize bringing products to market over securing them. This rush to release products often leaves security as an afterthought. Similar to the connected vehicle ecosystem, municipal drone systems involve multiple points of connectivity, which must all be secured to prevent potential breaches.

The drone industry is still in its early stages, and regulations surrounding their use, such as flying beyond line of sight (BLOS) and handling rogue drones, are still evolving. Integrating drones into complex processes like evidence collection also presents unresolved legal and procedural issues. Additionally, the industry is moving towards automation, with swarms of drones working together for various missions. However, the algorithms driving these capabilities are new and unproven, raising concerns about their potential security vulnerabilities.

As airspace becomes more crowded, issues such as drone interference with firefighting missions and other smart city operations will increase, necessitating the development of robust collision avoidance technologies. With the adoption of BLOS features, questions of liability and drone control also arise. Moreover, identifying rogue drone operators is becoming more difficult as waypoint planning and other advanced technologies are adopted.

The threats to drone systems range from privacy enthusiasts shooting down rogue drones to malicious actors attempting to compromise the command and control links between ground stations and drones. Techniques like GPS spoofing remain a significant threat, and the software controlling ground stations must be protected to prevent unauthorized access to the entire drone network.

Hacking a consumer drone typically involves monitoring the radio frequency spectrum, where many drones operate on frequencies like 900 MHz or 2.5 GHz. By identifying the telemetry channel and analyzing the data link, attackers may be able to intercept or even inject packets into the system. While this method doesn't apply to all drones and is not simple, it represents a potential avenue for security research.

In terms of municipal drone systems, the primary security goals are to ensure the safety of citizens and address privacy concerns. Since drones can cause physical harm, it is important to establish safeguards to prevent

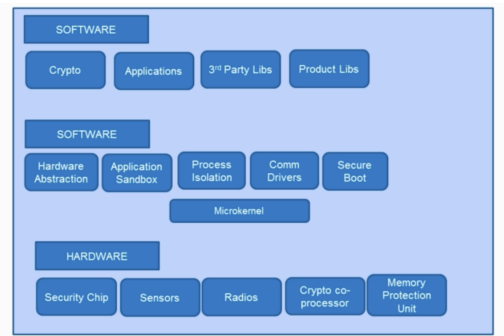


Figure 23: IoT architecture

accidents or injuries. Additionally, protecting sensitive data such as imagery and audio collected by drones is essential to maintaining public trust. Compliance with laws and regulations is also a key aspect of drone system security.

Establishing a secure municipal drone system involves several steps, including developing a threat model, conducting system-wide penetration tests, and updating incident isolation and response plans. It's also important to secure data at rest, both on drone platforms and ground stations, perform privacy and safety impact assessments, and maintain secure communications between drones and their controllers. Keeping drone software up to date is another critical aspect of maintaining a secure system.

As smart cities increasingly adopt drone technology, municipalities must carefully consider these security challenges. By addressing them through robust planning and ongoing monitoring, cities can ensure that their drone systems remain safe, secure, and ready for deployment at all times.

7.4 IoT hardware and software

IoT devices are composed of multiple layers, including hardware, operating system software, and application software. At the hardware layer, IoT devices typically use microprocessors based on ARM, MIPS, or x86 architectures. These devices also include sensors like temperature or pressure sensors, as well as radios to enable wireless communication over RF (Radio Frequency) spectrum. Security hardware, such as cryptographic co-processors or security chips, is often integrated to protect cryptographic operations used in higher layers of the technology stack.

At the operating system layer, many IoT devices use a real-time operating system (RTOS), which includes a microkernel and hardware abstraction layer. These systems provide key capabilities like communication drivers, process isolation, secure boot, and sandboxing for applications. The application

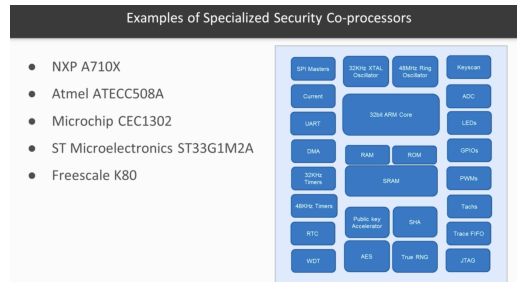


Figure 24: Examples for specialized microprocessor

layer involves custom applications, cryptographic protocols, and third-party libraries, often bundled into an image that is directly written onto the hardware. When updates are needed, the entire image is typically replaced.

It's crucial to note that the hardware selection for IoT devices impacts their security. Vulnerabilities can arise from hardware itself, and there are numerous tools that attackers can use to exploit these weaknesses. Developers may unintentionally expose IoT devices to threats by leaving debug interfaces open or allowing unauthorized access. A key security measure is ensuring that firmware is cryptographically verified each time it is updated or loaded.

Most IoT devices are built on System-on-a-Chip (SoC) architectures, which include microcontrollers to handle data collection, processing, and storage. When choosing hardware, developers should consider several factors, such as secure bootloaders to prevent malicious firmware from being loaded into memory and cryptographic accelerators to enhance processing efficiency. Tamper protection measures are also critical to prevent physical access to internal components, and managing cryptographic key materials securely is essential.

Key recommendations for securing IoT devices include using cryptographic hardware designed to address specific security risks, such as tamper detection systems. Tampering refers to unauthorized interference or manipulation of a device's hardware or software. To counter this, tamper detection systems can automatically erase sensitive data if such interference is detected, ensuring the device remains secure. Trusted bootloaders are also vital, as they check digital signatures to verify the software before starting the device, protecting it from unauthorized changes. Additionally, cryptographic co-processors, like the Microchip SEC1302, help handle encryption efficiently and securely store encryption keys. Implementing these measures strengthens the device's protection against growing security threats in IoT systems.

On the software side, IoT programs are often written in languages like Java, Python, C, C++, Go, Rust, and Node.js. IoT frameworks simplify

the development process by providing standardized services, including device onboarding, network provisioning, and credential management. Frameworks can also handle asset management, tracking, and secure firmware updates. A key feature of IoT frameworks is discovery services, which help devices locate peers and necessary servers for communication.

Another important aspect of IoT devices is the operating system. Most IoT devices run a scaled-down OS known as an RTOS, which handles task scheduling and hardware abstraction. Some RTOS implementations, like FreeRTOS and MicroLinux, are optimized for performance in consumer applications, while others, like VxWorks and LynxOS, are used in security-critical environments.

In summary, the hardware and software choices in IoT device development greatly impact the security and functionality of these systems. Developers need to carefully evaluate their hardware, software, and frameworks to ensure a secure and efficient IoT ecosystem. The next step in this discussion will focus on the communication protocols that enable IoT devices to interact with each other and cloud services.

7.5 IoT Communication and Messaging Protocols

IoT devices are interconnected systems designed to collect, transmit, and process data. These devices form networks that can collaborate to achieve various objectives. For instance, they might work together (communicate) over large areas, such as city-wide scale to manage infrastructure like smart grids or transportation systems. Alternatively, they could operate within more localized environments, such as homes or factories, to automate tasks or optimize processes.

When exploring IoT communication and messaging protocols, it's important to understand the various methods IoT devices use to connect and share information. Some devices rely on gateways to aggregate data at the network edge, while others may communicate directly with the cloud using services like Amazon's AWS. Another essential concept is the Wireless Sensor Network (WSN), which consists of lightweight communication protocols that connect devices to gateways and cloud services. These protocols can be adapted to allow dynamic, on-the-move communication.

In IoT environments such as connected vehicles, ensuring that messages are authentic and unaltered is critical. In technical terms, this means ensuring that messages are authenticated and that their integrity is protected. Without these protections, vehicles could receive false data, which could lead to dangerous situations, especially for systems that aid drivers or control autonomous navigation. One protocol used in vehicle-to-vehicle communication is DSRC (Dedicated Short-Range Communication), but numerous protocols exist for

IoT devices, each with its own security features.

Another example is the LTE (4G cellular) protocol, which secures communication using symmetric key cryptography. LTE networks use devices like smartphones equipped with UIM cards, which store authentication data. This data is used to generate secure keys for encrypting communications. Similarly, Bluetooth Low Energy (BLE) protocols employ security modes that range from minimal protection (such as "Just Works" pairing) to more secure methods requiring PIN codes or cryptographic keys for short-range communications. BLE security protocols follow a multi-step pairing process to ensure secure connections and protect against eavesdropping.

As IoT evolves, protocols like 5G promise to interconnect billions of devices, from drones to autonomous cars. Each protocol comes with its own set of security requirements and concerns, making it essential to use secure communication and messaging protocols in the growing IoT landscape.

7.6 Threats, Vulnerabilities and Risks in IoT

In this section, we will explore key concepts related to threats, vulnerabilities, and risks in IoT and IT security, starting with the fundamentals of information assurance.

Confidentiality is crucial for protecting sensitive data from unauthorized access or eavesdropping. One of the primary tools for achieving confidentiality is encryption, which transforms readable data (known as plaintext) into an unintelligible format (called ciphertext). Only individuals with the correct decryption key can access the original data.

Next, we have **integrity**, which ensures that data remains unchanged and unaltered during transmission or storage. This is often achieved using hash algorithms, like SHA (Secure Hash Algorithm), that produce a unique string (hash) based on the input data. If even a single character in the original data changes, the resulting hash will be entirely different, signaling potential tampering.

Authentication verifies the identity of those sending or receiving data. One effective method is the use of digital signatures, which combine hashing and encryption to ensure both the integrity and authenticity of a message. This means the recipient can be confident that the message genuinely comes from the claimed sender.

Another important concept is **non-repudiation**, which ensures that an individual cannot deny having taken an action, such as sending a message. This involves both technical mechanisms, like digital signatures, and procedural safeguards.

Availability refers to ensuring that systems are operational and accessible when needed. Threats to availability, such as Denial of Service (DoS) attacks, can disrupt services by overwhelming systems with traffic.

When discussing system security, it's essential to understand system compromise. This often involves exploiting a vulnerability—a weakness in the system or its environment. Threat actors, which can include hackers or insiders, identify these vulnerabilities to execute attacks.

Threat modeling is a critical process for identifying potential threats and developing strategies to mitigate them. It helps engineers anticipate and prepare for various security challenges. Following this, risk management assesses the likelihood and potential impact of identified risks. By prioritizing the most significant risks, organizations can allocate resources effectively to enhance security.

In the realm of IoT security, understanding these principles is vital, especially since many IoT devices operate for extended periods, making them susceptible to new vulnerabilities over time. A notable case study is the Mirai botnet, which took advantage of numerous unsecured IoT devices to launch widespread cyberattacks. This highlights the importance of applying robust security principles and comprehensive risk management strategies to protect both IT and IoT systems against evolving threats.

7.7 IoT Security Architecture

The concept of an IoT security architecture enables system designers to consider multiple layers of defense techniques tailored to their unique implementations. To effectively understand the components of an IoT system, we can refer to the U.S. National Institute of Standards and Technology (NIST), which identifies five core components:

- **Sensors:** These devices gather data and can vary significantly in capability, such as basic pressure sensors versus advanced connected vehicles.
- **Communications Channels:** This includes various types of connections, such as personal area networks, cellular networks, Wi-Fi, or Bluetooth.
- **Aggregators:** Serving as gateways, these devices collect data from sensors and communicate with the cloud, processing the data into appropriate formats.
- **Cloud Services:** The cloud provides essential functions like storage, messaging, analytics, and data processing.

- **Decision Triggers:** This aspect represents the logic that drives the system’s intended actions, such as using event-based actions in cloud services.

By understanding these components, designers can develop robust security architectures. The Industrial Internet Consortium outlines various views—business, usage, functional, and implementation—that aid in categorizing controls based on system concerns such as safety, security, trust, and privacy.

For example, the link between safety and security is crucial, especially in cyber-physical systems like connected cars or medical devices, where security compromises can lead to severe consequences. Resilience must also be considered to ensure the system can withstand attacks.

Additionally, integrating security measures should be feasible throughout the system’s lifecycle, enabling updates and enhancements. Specific controls at different architecture levels, such as whitelisting trusted communications and enforcing strong authentication methods, are vital for protecting edge devices and cloud gateways.

This overview sets the foundation for a deeper exploration of IoT security architectures.

7.8 Lightweight Encryption Algorithms for IoT

Lightweight cryptographic techniques are essential in Internet of Things (IoT) environments, where devices often operate with limited processing power, memory, and battery life. To secure communication under these constraints, three main categories of symmetric cryptography are used: block ciphers, stream ciphers, and AEAD (Authenticated Encryption with Associated Data). Each serves a different purpose and offers unique advantages depending on the application and device capabilities.

7.8.1 Block Ciphers - “Encrypt in chunks”

Block ciphers are symmetric-key cryptographic algorithms that encrypt data in fixed-size blocks, such as 64 or 128 bits. In other words: Block ciphers take your data, chop it into equal-sized pieces (called blocks), and encrypt each piece one at a time. Think of them like sealing information into a series of small boxes.

They rely on repeated rounds of substitution, permutation, and key mixing to transform plaintext into ciphertext. In lightweight cryptography for IoT, block ciphers are designed to use minimal hardware resources, small memory footprints, and low power consumption while still resisting modern cryptanalytic

attacks. Their deterministic structure makes them reliable for environments where predictable performance and hardware efficiency are essential, such as RFID tags, embedded controllers, and sensor nodes. However, because block ciphers operate on fixed-size blocks, they may require additional modes of operation to handle variable-length messages or provide authentication.

List of Block ciphers algorithms

Example: A smart door lock encrypts its 64-byte configuration file before storage. Since the data is fixed and relatively small, a block cipher can efficiently encrypt each block of the file and keep the device secure without requiring much processing power.

Super simple example: You want to store a small file on a smart device. The file is split into blocks, and each block is locked with the same key.

7.8.2 Stream Ciphers - "Encrypt as you go"

Stream ciphers encrypt data continuously, one bit or one byte at a time, as the data flows. They're like a pipe where data goes in, and encrypted data comes out instantly. They encrypt data one bit or byte at a time by generating a pseudorandom keystream that is combined with the plaintext. This makes them highly suitable for low-latency or real-time communication, especially in constrained IoT systems where memory and computational capabilities are very limited. Stream ciphers often have simpler internal structures compared to block ciphers, allowing them to run efficiently on small microcontrollers or dedicated hardware. Their continuous keystream generation enables them to handle data of arbitrary length without extra modes, but they require strict management of keys and nonces to avoid keystream reuse, which can compromise security.

Example: A wireless temperature sensor sends readings every second, each reading is encrypted immediately as it's produced. A stream cipher encrypts each byte of data on the fly, allowing the device to transmit securely with almost no delay and minimal energy consumption.

7.8.3 AEAD (Authenticated Encryption with Associated Data) - "Encrypt and verify"

AEAD algorithms provide both encryption and authentication in a single, integrated process. This ensures not only confidentiality but also integrity and authenticity—critical properties for IoT systems operating in potentially hostile environments. AEAD reduces system complexity by eliminating the need for separate encryption and message authentication components, which

saves memory and power while offering modern security guarantees. AEAD algorithms combine encryption and authentication in a single operation, ensuring both confidentiality and integrity of messages. In IoT environments, where devices may be exposed to active attackers and unreliable networks, AEAD provides strong protection by preventing tampering, replay, and forgery attacks. Unlike basic encryption, AEAD produces both ciphertext and an authentication tag, which allows the receiver to verify that the data has not been modified. Lightweight AEAD constructions are particularly important for IoT because they eliminate the need for separate encryption and message authentication components, reducing memory use, power consumption, and code complexity while offering robust security for modern connected devices.

Example: A medical wearable sends a patient's heart rate data to a monitoring station. AEAD ensures the data is encrypted and comes with an authentication tag, so the receiver can confirm that no attacker modified or injected false measurements.